



Εθνικό Μέτσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 6ο : Βάσεις Δεδομένων

Διδάσκοντες: Δ. Τσουμάκος, Μ. Κονιαρης, Β. Στεφανίδης

Αναφορά Εξαμηνιαίας Εργασίας

“Σύστημα αποθήκευσης και διαχείρισης των πληροφοριών για γενικό Νοσοκομείο ”

Ομάδα εξαμηνιαίας εργασίας:	Ομάδα Project 99
Ονοματεπώνυμα:	Τσιατσιάνης Ιωάννης (ΑΜ: 03121812) Ευταξιώπουλος Παναγιώτης (ΑΜ: 03122064) Φραντζεσκάκης Μιχαήλ (ΑΜ: 03122147)
Τίτλοι ασκήσεων:	1. Διάγραμμα ER & Σχεσιακό Διάγραμμα 2. Δημιουργία του σχήματος της βάσης δεδομένων 3. Φόρτωση του σχήματος με δεδομένα 4. Δημιουργία SQL queries
Ημερομηνία παράδοσης:	Δευτέρα 20 Μαΐου 2026

Περιεχόμενα

Διάγραμμα ER & Σχεσιακό Διάγραμμα:	2
Σχεδιαστικές Αποφάσεις και Παραδοχές:	2
Μοντέλο Οντοτήτων - Συσχετίσεων (ER Diagram)	2
Σχεσιακό Σχήμα (Relational Diagram)	3
Δημιουργία του σχήματος της βάσης δεδομένων (install.sql)	5
Φόρτωση του σχήματος με δεδομένα (load.sql)	8
Μεθοδολογία Παραγωγής Δεδομένων	8
Σειρά Εισαγωγής και Αναφορική Ακεραιότητα	8
Στοχευμένη Προσαρμογή Δεδομένων (Data Tweaking).....	8
Δημιουργία SQL queries.....	9
Ερώτημα 1 (Q1).....	9
Ερώτημα 2 (Q2).....	11
Ερώτημα 3 (Q3).....	12
Ερώτημα 4 (Q4).....	13
Ερώτημα 5 (Q5).....	17
Ερώτημα 6 (Q6).....	18
Ερώτημα 7 (Q7).....	21
Ερώτημα 8 (Q8).....	22
Ερώτημα 9 (Q9).....	23
Ερώτημα 10 (Q10).....	25
Ερώτημα 11 (Q11).....	26
Ερώτημα 12 (Q12).....	27
Ερώτημα 13 (Q13).....	28
Ερώτημα 14 (Q14).....	30
Ερώτημα 15 (Q15).....	31

Διάγραμμα ER & Σχεσιακό Διάγραμμα:

Σχεδιαστικές Αποφάσεις και Παραδοχές:

Κατά τον σχεδιασμό του Εννοιολογικού Μοντέλου (ER) και τη μετάβασή του στο Σχεσιακό Μοντέλο, ελήφθησαν υπόψη οι πραγματικές ανάγκες λειτουργίας ενός Γενικού Νοσοκομείου. Οι βασικότερες σχεδιαστικές αποφάσεις και παραδοχές που καθόρισαν τη δομή της βάσης δεδομένων είναι οι εξής:

Υλοποιήθηκε μια ιεραρχική δομή υπερκλάσης-υποκλάσης για το προσωπικό. Η οντότητα Staff περιέχει τα κοινά στοιχεία (Ονοματεπώνυμο, Τηλέφωνο, Ημερομηνία Πρόσληψης κ.λπ.) και διακλαδίζεται στις υποκλάσεις Doctor, Nurse και Admin. Με αυτόν τον τρόπο αποφεύγεται ο πλεονασμός δεδομένων και διευκολύνεται η διαχείριση του ανθρώπινου δυναμικού.

Στην οντότητα Doctor προστέθηκε αναδρομική συσχέτιση (supervisor_AMKA) για την αποτύπωση της ιατρικής ιεραρχίας (π.χ. Ειδικευόμενος, Επιμελητής, Διευθυντής). Κάθε ιατρός μπορεί να επιβλέπει πολλούς άλλους, αλλά αναφέρεται σε ακριβώς έναν επόπτη (ή σε κανέναν, αν είναι στην κορυφή της ιεραρχίας).

Μια εισαγωγή (Admission) αποτελεί τον κεντρικό πυρήνα γύρω από τον οποίο περιστρέφονται οι ιατρικές πράξεις. Κάθε νοσηλεία συνδέεται υποχρεωτικά με έναν ασθενή (Patient), ένα συγκεκριμένο τμήμα (Department) και μία κλίνη (Bed). Για την εξασφάλιση της διαλειτουργικότητας και της σωστής τιμολόγησης, οι διαγνώσεις εισόδου και εξόδου συνδέονται με τον διεθνή κατάλογο ICD-10, ενώ η χρέωση βασίζεται στο σύστημα Κλειστών Ενοποιημένων Νοσηλίων (KEN).

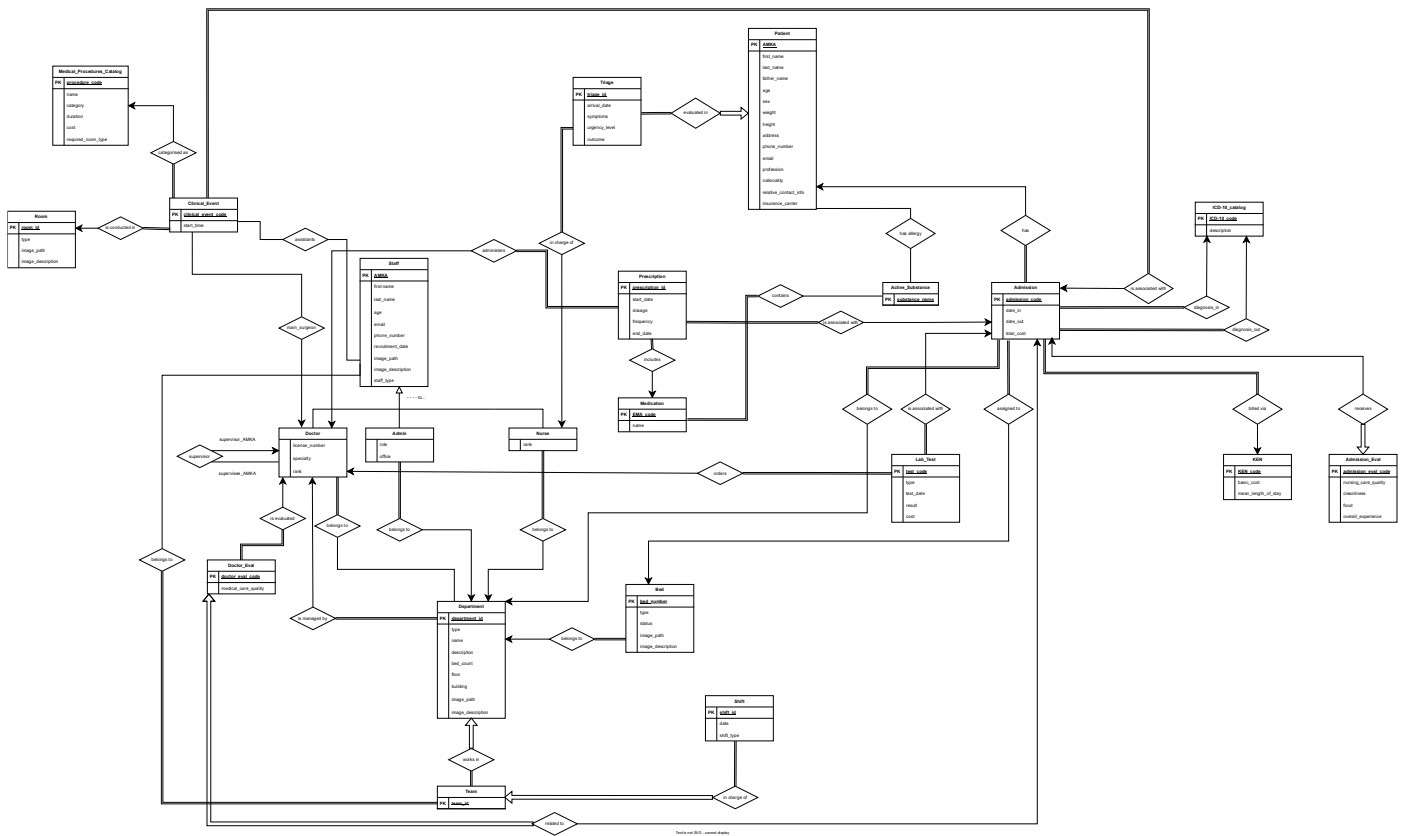
Για την καταγραφή χειρουργείων και εξετάσεων δημιουργήθηκε η οντότητα Clinical_Event, η οποία συνδέεται με έναν κύριο χειρουργό (main surgeon), έναν συγκεκριμένο χώρο/δωμάτιο (Room) και την τρέχουσα νοσηλεία. Η ύπαρξη του βοηθητικού πίνακα Clinical_event_assistants επιτρέπει την καταγραφή πολλαπλών βοηθών για το ίδιο συμβάν (σχέση N:M). Το σύστημα επιτρέπει την αξιολόγηση τόσο των παρεχόμενων υπηρεσιών φιλοξενίας (Admission_Eval) όσο και του ιατρικού προσωπικού (Doctor_Eval). Οι αξιολογήσεις είναι προαιρετικές, αλλά συνδέονται αυστηρά με έναν συγκεκριμένο κωδικό νοσηλείας (Admission Code) για να διασφαλίζεται η εγκυρότητά τους.

Η οργάνωση του χρόνου εργασίας δεν γίνεται σε ατομικό επίπεδο, αλλά ομαδικά. Το προσωπικό ανήκει σε Ομάδες (Team), και οι Ομάδες αναλαμβάνουν συγκεκριμένες Βάρδιες (Shift) σε συγκεκριμένα τμήματα, διευκολύνοντας τον προγραμματισμό (Scheduling).

Αναλυτικότερες παραδοχές υπάρχουν στο αρχείο readme που επισυνάπτεται με την εργασία. Για την υλοποίηση της εργασίας χρησιμοποιήθηκε gemini pro, για βοήθεια στην παραγωγή των dummy data και για review του κώδικα και κειμένου. Προκειμένου να επιτευχθεί η ταχύτερη εκτέλεση (query optimization) και η αποτελεσματικότερη προσπέλαση των δεδομένων στα πολύπλοκα ερωτήματα SQL, σχεδιάστηκαν και εφαρμόστηκαν κατάλληλα ευρετήρια (indexes) στους πίνακες της βάσης.

Μοντέλο Οντοτήτων - Συσχετίσεων (ER Diagram)

Στο παρακάτω σχήμα παρουσιάζεται το εννοιολογικό μοντέλο της βάσης δεδομένων (ER Diagram), το οποίο σχεδιάστηκε με βάση τις προδιαγραφές και τις λειτουργικές απαιτήσεις του πληροφοριακού συστήματος του νοσοκομείου. Για την υλοποίηση αξιοποιήθηκε το πρόγραμμα Draw.io.

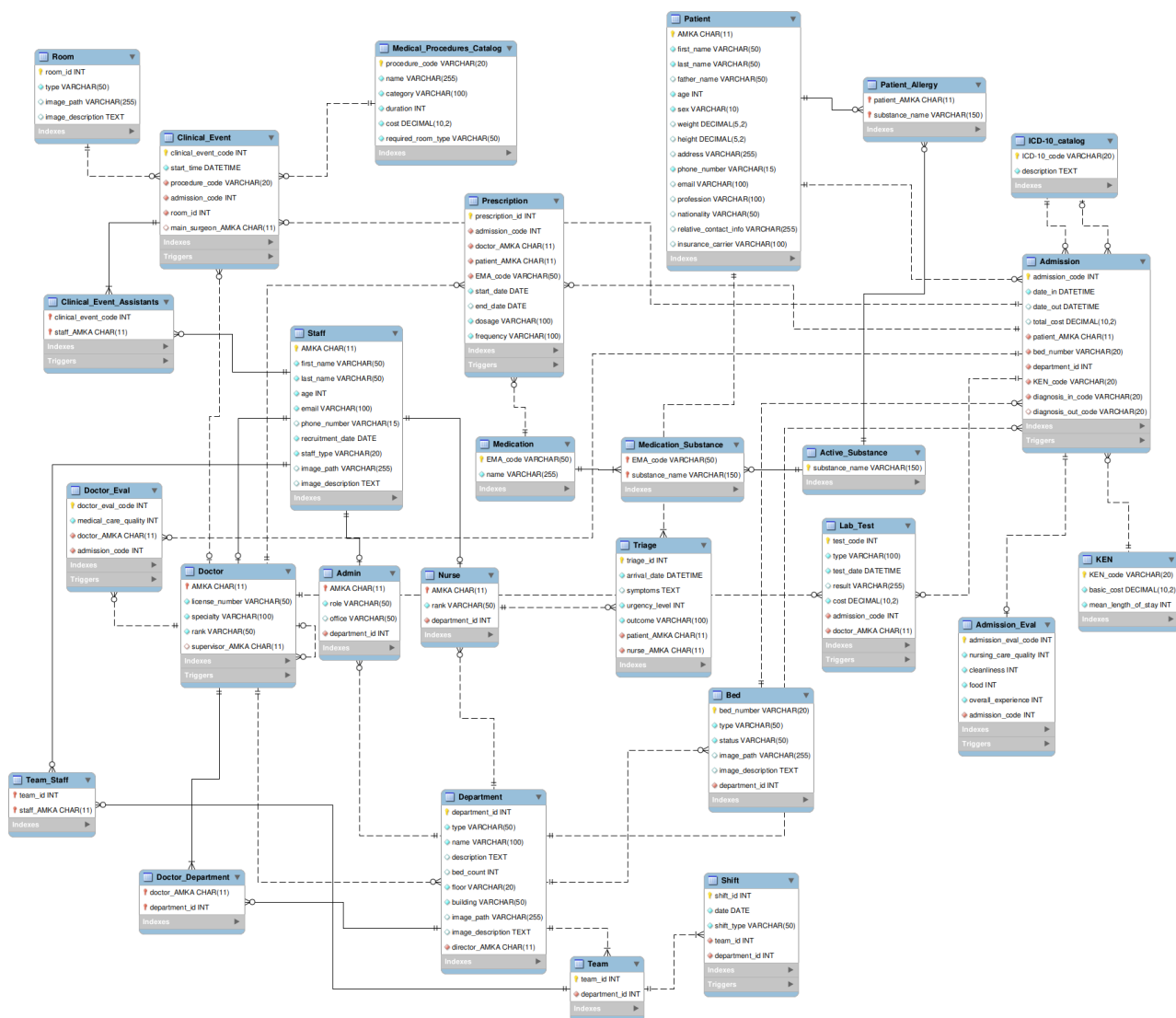


Σχεσιακό Σχήμα (Relational Diagram)

Βασιζόμενοι στο διάγραμμα Οντοτήτων-Συσχετίσεων (ER), προχωρήσαμε στον μετασχηματισμό του στο παρακάτω Σχεσιακό Σχήμα. *Σημείωση: Με έντονη γραφή και υπογράμμιση* συμβολίζονται τα Πρωτεύοντα Κλειδιά (Primary Keys), ενώ με πλάγια γραφή και αστερίσκο (*) συμβολίζονται τα Ξένα Κλειδιά (Foreign Keys).

- **patient** (*AMKA*, first_name, last_name, father_name, age, sex, weight, height, address, phone_number, email, profession, nationality, relative_contact_info, insurance_carrier)
- **staff** (*AMKA*, first_name, last_name, age, email, phone_number, recruitment_date, image_path)
- **doctor** (*AMKA*, license_number, specialty, rank, *supervisor_AMKA**)
- **nurse** (*AMKA*, rank, *department_id**)
- **admin** (*AMKA*, role, office, *department_id**)
- **department** (*department_id*, type, name, description, bed_count, floor, building, image_path, image_description, *director_AMKA**)
- **bed** (*bed number*, type, status, image path, image description, *department id**)

- **room** (**room_id**, type, image_path, image_description)
- **icd-10_catalog** (**ICD-10_code**, description)
- **ken** (**KEN_code**, basic_cost, mean_length_of_stay)
- **admission** (**admission_code**, date_in, date_out, total_cost, *patient_AMKA**, *bed_number**, *department_id**, *KEN_code**, *diagnosis_in_code**, *diagnosis_out_code**)
- **triage** (**triage_id**, arrival_date, symptoms, urgency_level, outcome, *patient_AMKA**, *nurse_AMKA**)
- **admission_eval** (**admission_eval_code**, nursing_care_quality, cleanliness, food, overall_experience, *admission_code**)
- **doctor_eval** (**doctor_eval_code**, medical_care_quality, *doctor_AMKA**, *admission_code**)
- **active_substance** (**substance_name**)
- **medication** (**EMA_code**, name)
- **medication_substance** (*EMA_code*, *substance_name*)
- **patient_allergy** (*patient_AMKA*, *substance_name*)
- **prescription** (**prescription_id**, *admission_code**, *doctor_AMKA**, *EMA_code**, start_date, end_date, dosage, frequency)
- **medical_procedures_catalog** (**procedure_code**, name, category, duration, cost, required_room_type)
- **clinical_event** (**clinical_event_code**, start_time, *procedure_code**, *admission_code**, *room_id**, *main_surgeon_AMKA**)
- **clinical_event_assistants** (*clinical_event_code*, *staff_AMKA*)
- **lab_test** (**test_code**, type, test_date, result, cost, *admission_code**, *doctor_AMKA**)
- **team** (**team_id**, *department_id**)
- **team_staff** (*team_id*, *staff_AMKA*)
- **shift** (**shift_id**, date, shift_type, *team_id**)



Εικόνα 2: Relational Diagram

Δημιουργία του σχήματος της βάσης δεδομένων (install.sql)

Για τη φυσική υλοποίηση (Physical Design) της βάσης δεδομένων επιλέχθηκε το Σύστημα Διαχείρισης Σχεσιακών Βάσεων Δεδομένων (RDBMS) MySQL / MariaDB. Η δημιουργία του σχήματος (schema) και των πινάκων πραγματοποιήθηκε με χρήση της γλώσσας ορισμού δεδομένων (Data Definition Language - DDL).

Κατά τη συγγραφή των εντολών CREATE TABLE, δόθηκε ιδιαίτερη έμφαση στη διασφάλιση της ακεραιότητας των δεδομένων (Data Integrity) μέσω των εξής τεχνικών:

- **Πρωτεύοντα Κλειδιά (Primary Keys):** Ορίστηκαν σε όλους τους πίνακες για τη μοναδική ταυτοποίηση της κάθε εγγραφής (π.χ. AMKA στους ασθενείς, admission_code στις εισαγωγές). Στις περιπτώσεις αναγνωριστικών που παράγονται από το σύστημα (surrogate keys), χρησιμοποιήθηκε η ιδιότητα AUTO_INCREMENT.

- Ξένα Κλειδιά (Foreign Keys): Υλοποιήθηκαν οι συσχετίσεις μεταξύ των πινάκων, επιβάλλοντας αναφορική ακεραιότητα (Referential Integrity). Όπου κρίθηκε απαραίτητο, ορίστηκαν κανόνες διαγραφής και ενημέρωσης (π.χ. ON DELETE RESTRICT ή ON DELETE CASCADE) για την αποφυγή ορφανών εγγραφών.
- Περιορισμοί Πεδίων (Constraints): Χρησιμοποιήθηκαν περιορισμοί όπως NOT NULL σε υποχρεωτικά πεδία (π.χ. ονόματα, ημερομηνίες) και UNIQUE σε πεδία που δεν πρέπει να επαναλαμβάνονται (π.χ. email, αριθμός άδειας ασκήσεως επαγγέλματος).
- Τύποι Δεδομένων (Data Types): Επιλέχθηκαν οι βέλτιστοι τύποι δεδομένων για εξοικονόμηση χώρου και σωστή αναπαράσταση της πληροφορίας (π.χ. DECIMAL για κόστη και μετρήσεις, DATETIME για ακριβή καταγραφή γεγονότων, και ENUM για πεδία με προκαθορισμένες τιμές όπως το rank).

Ενδεικτικά παρατίθεται κομμάτι από το SQL Script με τις εντολές DDL που εκτελέστηκαν για τη δημιουργία του σχήματος:

-- Δημιουργία ανεξάρτητων πινάκων (Λεξικά και Κατάλογοι)

```
DROP DATABASE IF EXISTS Ergasiav8;
CREATE DATABASE Ergasiav8;
USE Ergasiav8;
```

```
CREATE TABLE `Active_Substance` (
  `substance_name` VARCHAR(150) NOT NULL,
  PRIMARY KEY (`substance_name`)
);
```

```
CREATE TABLE `ICD-10_catalog` (
  `ICD-10_code` VARCHAR(20) NOT NULL,
  `description` TEXT NOT NULL, -- Η περιγραφή είναι απαραίτητη
  PRIMARY KEY (`ICD-10_code`)
);
```

```
CREATE TABLE `KEN` (
  `KEN_code` VARCHAR(20) NOT NULL,
  `basic_cost` DECIMAL(10,2) NOT NULL,
  `mean_length_of_stay` INT NOT NULL,
  PRIMARY KEY (`KEN_code`)
);
```

```
CREATE TABLE `Medical_Procedures_Catalog` (
  `procedure_code` VARCHAR(20) NOT NULL,
  `name` VARCHAR(255) NOT NULL,
  `category` VARCHAR(100) NOT NULL,
  `duration` INT NOT NULL, -- Διάρκεια σε λεπτά
  `cost` DECIMAL(10,2) NOT NULL,
  `required_room_type` VARCHAR(50) NOT NULL,
  PRIMARY KEY (`procedure_code`),
  CONSTRAINT `chk_category` CHECK (`category` IN ('Surgery', 'Diagnostic', 'Therapeutic')),
```

```

    CONSTRAINT `chk_required_room_type` CHECK (`required_room_type` IN ('Operating',
'Procedure'))
);

```

```

CREATE TABLE `Medication` (
  `EMA_code` VARCHAR(50) NOT NULL,
  `name` VARCHAR(255) NOT NULL,
  PRIMARY KEY (`EMA_code`)
);

```

```

CREATE TABLE `Room` (
  `room_id` INT NOT NULL AUTO_INCREMENT,
  `type` VARCHAR(50) NOT NULL,
  `image_path` VARCHAR(255),
  `image_description` TEXT,
  PRIMARY KEY (`room_id`),
  CONSTRAINT `chk_room_type` CHECK (`type` IN ('Operating', 'Procedure'))
);

```

-- Δημιουργία βασικών οντοτήτων (Patient & Staff)

```

CREATE TABLE `Patient` (
  `AMKA` CHAR(11) NOT NULL,
  `first_name` VARCHAR(50) NOT NULL,
  `last_name` VARCHAR(50) NOT NULL,
  `father_name` VARCHAR(50),
  `age` INT NOT NULL,
  `sex` VARCHAR(10) NOT NULL,
  `weight` DECIMAL(5,2),
  `height` DECIMAL(5,2),
  `address` VARCHAR(255),
  `phone_number` VARCHAR(15) NOT NULL,
  `email` VARCHAR(100),
  `profession` VARCHAR(100),
  `nationality` VARCHAR(50),
  `relative_contact_info` VARCHAR(255),
  `insurance_carrier` VARCHAR(100),
  PRIMARY KEY (`AMKA`)
);

```

```

CREATE TABLE `Staff` (
  `AMKA` CHAR(11) NOT NULL,
  `first_name` VARCHAR(50) NOT NULL,
  `last_name` VARCHAR(50) NOT NULL,
  `age` INT NOT NULL,
  `email` VARCHAR(100) NOT NULL,
  `phone_number` VARCHAR(15),
  `recruitment_date` DATE NOT NULL,
  `staff_type` VARCHAR(20) NOT NULL,
  `image_path` VARCHAR(255),
  `image_description` TEXT,
  PRIMARY KEY (`AMKA`),
  UNIQUE (`email`), -- Κάθε υπάλληλος έχει μοναδικό email

```



```
CONSTRAINT `chk_staff_type` CHECK (`staff_type` IN ('Doctor', 'Nurse', 'Admin'))
);

CREATE INDEX idx_age ON `Staff` (`age`);
```

Φόρτωση του σχήματος με δεδομένα (load.sql)

Η εισαγωγή εικονικών δεδομένων (mock data) στη βάση αποτέλεσε ένα κρίσιμο στάδιο για την επαλήθευση της ορθότητας του σχεδιασμού και την αξιολόγηση της απόδοσης των πολύπλοκων ερωτημάτων (SQL queries). Λόγω του μεγάλου όγκου των απαιτούμενων εγγραφών, η φόρτωση δεν πραγματοποιήθηκε χειροκίνητα, αλλά μέσω αυτοματοποιημένων SQL scripts.

Μεθοδολογία Παραγωγής Δεδομένων

Για τη δημιουργία ρεαλιστικών δεδομένων, αξιοποιήθηκαν οι ενσωματωμένες μαθηματικές συναρτήσεις και συναρτήσεις συμβολοσειρών του RDBMS (όπως RAND(), FLOOR(), και ELT()). Με τον τρόπο αυτό επιλέχθηκαν τυχαίοι συνδυασμοί ονομάτων και επιθέτων για το προσωπικό και τους ασθενείς. Δημιουργήθηκαν ημερομηνίες (π.χ. εισαγωγής, εξιτηρίου, άφιξης στο ΤΕΠ) διεσπαρμένες σε ένα εύρος ετών (2021-2025), ώστε να είναι δυνατή η εξαγωγή στατιστικών και χρονικών αναλύσεων. Κατανεμήθηκαν τυχαία αλλά ρεαλιστικά κόστη, βαθμολογίες αξιολογήσεων (1-5) και κωδικοί διαγνώσεων (ICD-10) στις αντίστοιχες εισαγωγές.

Σειρά Εισαγωγής και Αναφορική Ακεραιότητα

Λόγω του αυστηρού σχεσιακού μοντέλου και των περιορισμών των περιορισμών που υλοποιήσαμε, η εισαγωγή των δεδομένων έγινε με αυστηρά ιεραρχική σειρά (Top-Down Approach):

1. **Λεξικά και Κατάλογοι (Independent Tables):** Αρχικά φορτώθηκαν οι πίνακες που δεν εξαρτώνται από άλλους, όπως οι κατάλογοι ICD-10_catalog, KEN, Medical_Procedures_Catalog, Room, Active_Substance και Medication.
2. **Βασικές Οντότητες (Core Entities):** Ακολούθησε η εισαγωγή σταθερών οντοτήτων, όπως τα τμήματα (Department), τα κρεβάτια (Bed), το ανθρώπινο δυναμικό (Staff και οι υποκλάσεις του Doctor, Nurse, Admin) και οι ασθενείς (Patient). Ιδιαίτερη προσοχή δόθηκε στην αναδρομική σχέση των ιατρών, όπου πρώτα ορίστηκαν οι Διευθυντές (χωρίς επόπτη) και στη συνέχεια οι υφιστάμενοί τους.
3. **Οντότητες Γεγονότων και Συσχετίσεων (Transactional & Associative Entities):** Τέλος, φορτώθηκαν οι πίνακες που καταγράφουν τη "ζωή" του νοσοκομείου και ενώνουν τα παραπάνω δεδομένα, όπως οι εισαγωγές (Admission), τα κλινικά συμβάντα (Clinical_Event), οι εφημερίες/βάρδιες (Shift) και οι αξιολογήσεις (Admission_Eval, Doctor_Eval).

Στοχευμένη Προσαρμογή Δεδομένων (Data Tweaking)

Για να ελεγχθούν επαρκώς οι ακραίες περιπτώσεις (edge cases) των πολύπλοκων ερωτημάτων SQL, εφαρμόστηκαν στοχευμένες ενημερώσεις (UPDATE) στα τυχαία δεδομένα. Χαρακτηριστικά

παραδείγματα αποτελούν η τεχνητή δημιουργία "τακτικών" ασθενών με πολλαπλές νοσηλείες στο ίδιο τμήμα, για την επαλήθευση ερωτημάτων ομαδοποίησης (GROUP BY ... HAVING). Επιπλέον επιτεύχθηκε ο χρονικός συγχρονισμός μεταξύ της άφιξης στο Τμήμα Επειγόντων Περιστατικών (Triage) και της Εισαγωγής (Admission), ώστε να εμπίπτουν σε ρεαλιστικά παράθυρα αναμονής 24 ωρών και να υπολογίζονται σωστά οι μέσοι χρόνοι αναμονής ανά επίπεδο επείγοντος.

Δημιουργία SQL queries

Στο παρόν κεφάλαιο παρουσιάζεται η ανάλυση και η εξαγωγή χρήσιμης πληροφορίας από τη βάση δεδομένων του νοσοκομείου, μέσω δεκαπέντε (15) στοχευμένων ερωτημάτων SQL. Τα ερωτήματα σχεδιάστηκαν έτσι ώστε να καλύπτουν ρεαλιστικά σενάρια διοικητικής, οικονομικής και ιατρικής φύσεως (π.χ. στατιστικά νοσηλείων, ιεραρχία προσωπικού, αξιολογήσεις, διαχείριση ΤΕΠ). Για την υλοποίησή τους αξιοποιήθηκαν προχωρημένες δυνατότητες της SQL, όπως:

- Common Table Expressions (CTEs) & Αναδρομικά CTEs: Για την απλοποίηση πολύπλοκων ερωτημάτων και την αναπαράσταση ιεραρχικών δομών (π.χ. δέντρο εποπτείας ιατρών).
- Πολλαπλές Συνενώσεις (JOINS): Εσωτερικές (INNER) και εξωτερικές (LEFT/RIGHT) συνενώσεις για τη σύνδεση κατανεμημένης πληροφορίας από πολλαπλούς πίνακες.
- Συναρτήσεις Ομαδοποίησης και Φιλτραρίσματος: Χρήση GROUP BY σε συνδυασμό με HAVING και συναρτήσεις συνάθροισης (COUNT, SUM, AVG) για την εξαγωγή στατιστικών δεικτών.
- Window Functions: Για τον υπολογισμό κυλιόμενων αθροισμάτων και κατατάξεων χωρίς την απώλεια της αναλυτικής πληροφορίας των γραμμών.
- Self-Joins: Για τη σύγκριση δεδομένων του ίδιου πίνακα (π.χ. σύγκριση στατιστικών ασθενειών μεταξύ συνεχόμενων ετών).

Για κάθε ερώτημα παρατίθεται η εκφώνηση (ο επιχειρησιακός στόχος), ο αντίστοιχος κώδικας SQL και ένα ενδεικτικό απόσπασμα των αποτελεσμάτων που επιστρέφει το Σύστημα Διαχείρισης Βάσης Δεδομένων.

Ερώτημα 1 (Q1)

Βρείτε τα συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά ΚΕΝ κωδικό (βασικό κόστος έναντι πρόσθετης χρέωσης λόγω υπέρβασης ΜΔΝ) και κατανομή νοσηλείων ανά ασφαλιστικό φορέα.

USE Ergasiav8;

```

SELECT
    d.name AS Department_Name,
    YEAR(a.date_in) AS Admission_Year,
    a.KEN_code AS KEN_Code,
    p.insurance_carrier AS Insurance_Carrier,
    COUNT(a.admission_code) AS Total_Admissions,
    SUM(k.basic_cost) AS Total_Basic_Cost_KEN,
    SUM(
        GREATEST(0, GREATEST(1, CEILING(TIMESTAMPDIFF(HOUR, a.date_in,
        COALESCE(a.date_out, NOW())) / 24)) - k.mean_length_of_stay)
        * (k.basic_cost / NULLIF(k.mean_length_of_stay, 0))
    ) AS Total_Additional_Charge,
    SUM(a.total_cost) AS Total_Revenue
FROM
    Admission a
JOIN
    Department d ON a.department_id = d.department_id
JOIN
    Patient p ON a.patient_AMKA = p.AMKA
JOIN
    KEN k ON a.KEN_code = k.KEN_code
GROUP BY
    d.name,
    YEAR(a.date_in),
    a.KEN_code,
    p.insurance_carrier
ORDER BY
    Department_Name ASC,
    Admission_Year DESC,
    KEN_Code ASC,
    Total_Revenue DESC;

```

Αποτέλεσμα:

	Department_Name	Admission_Year	KEN_Code	Insurance_Carrier	Total_Admissions	Total_Basic_Cost_KEN	Total_Additional_Charge	Total_Revenue
	Γαστρεντερολογικό Τμήμα	2023	N24M	Ανασφάλιστος	1	2389.00	196449.307692	200267.71
	Γαστρεντερολογικό Τμήμα	2023	N30A	ΤΥΠΕΤ	1	420.00	1050.000000	2587.47
	Γαστρεντερολογικό Τμήμα	2023	N34X	Ανασφάλιστος	1	1012.00	218794.400000	220544.77
	Γαστρεντερολογικό Τμήμα	2023	Y21A	Ανασφάλιστος	1	231.00	236082.000000	237255.54
	Δερματολογικό Τμήμα	2025	Θ21A	ΕΦΚΑ	1	2599.00	0.000000	3443.20
	Δερματολογικό Τμήμα	2025	N24M	ΤΥΠΕΤ	1	2389.00	0.000000	3208.41
	Δερματολογικό Τμήμα	2024	Π46A	Ιδιωτική Ασφάλεια	1	327.00	981.000000	2547.03
	Δερματολογικό Τμήμα	2023	S25A	Ιδιωτική Ασφάλεια	1	1082.00	1082.000000	3213.15
	Δερματολογικό Τμήμα	2023	Γ07A	ΕΦΚΑ	1	742.00	746452.000000	748741.07
	Δερματολογικό Τμήμα	2023	E12A	ΤΥΠΕΤ	1	6414.00	2058894.000000	2066835.97
	Δερματολογικό Τμήμα	2023	H42M	ΕΦΚΑ	1	1938.00	237405.000000	240137.65
	Δερματολογικό Τμήμα	2023	M23A	Ανασφάλιστος	1	661.00	5949.000000	7990.18
	Δερματολογικό Τμήμα	2023	Φ64X	Ιδιωτική Ασφάλεια	1	303.00	272094.000000	273019.44
	Δερματολογικό Τμήμα	2023	Ω31A	ΕΦΚΑ	1	444.00	666.000000	2809.44
	Ενδοκρινολογικό Τμήμα	2025	S23M	ΟΠΑΔ	1	2039.00	0.000000	2706.60
	Ενδοκρινολογικό Τμήμα	2025	Φ04X	Ιδιωτική Ασφάλεια	1	5558.00	196045.818182	203332.88
	Ενδοκρινολογικό Τμήμα	2024	K18M	Ανασφάλιστος	1	2226.00	1272.000000	3911.17
	Ενδοκρινολογικό Τμήμα	2024	M13X	ΤΥΠΕΤ	1	2083.00	3645.250000	6522.72
	Ενδοκρινολογικό Τμήμα	2024	M76X	ΤΥΠΕΤ	1	389.00	1167.000000	3098.33
	Ενδοκρινολογικό Τμήμα	2023	G01Mβ	ΕΦΚΑ	1	3573.00	397.000000	5333.34
	Ενδοκρινολογικό Τμήμα	2023	Δ12M	Ιδιωτική Ασφάλεια	1	4871.00	0.000000	5715.86
	Ενδοκρινολογικό Τμήμα	2023	E12A	ΤΥΠΕΤ	1	6414.00	10690.000000	17982.91
	Ενδοκρινολογικό Τμήμα	2023	Θ23X	Ιδιωτική Ασφάλεια	1	405.00	810.000000	1696.31

Σχόλια:

Το ερώτημα πραγματοποιεί συνενώσεις (JOIN) μεταξύ 4 κεντρικών πινάκων (Admission, Department, Patient, KEN) και ομαδοποιεί τα δεδομένα σε 4 επίπεδα (Τμήμα, Έτος, Κωδικό KEN, Ασφαλιστικό Φορέα), προσφέροντας μια εξαιρετικά λεπτομερή (granular) ανάλυση. Χρησιμοποιείται η συνάρτηση TIMESTAMPDIFF(HOUR, ...) σε συνδυασμό με την CEILING(... / 24) για τη μετατροπή των ωρών νοσηλείας σε ακέραιες ημέρες (στρογγυλοποίηση προς τα πάνω), διασφαλίζοντας ότι ακόμα και μια ολιγόωρη παραμονή θα καταγραφεί ως τουλάχιστον 1 ημέρα μέσω της GREATEST(1, ...). Παράλληλα, η COALESCE(a.date_out, NOW()) προστατεύει το ερώτημα σε περιπτώσεις ενεργών νοσηλίων όπου δεν έχει οριστεί ακόμα ημερομηνία εξιτηρίου. Η λογική της πρόσθετης χρέωσης λόγω υπέρβασης της Μέσης Διάρκειας Νοσηλείας (ΜΔΝ) υλοποιείται πλήρως δυναμικά. Αφαιρείται η ΜΔΝ (k.mean_length_of_stay) από τις πραγματικές ημέρες, και η εξωτερική GREATEST(0, ...) διασφαλίζει ότι αν ένας ασθενής πήρε εξιτήριο νωρίτερα ή ακριβώς στη ΜΔΝ, η πρόσθετη χρέωση θα παραμείνει μηδενική. Το γινόμενο πολλαπλασιάζεται με το ημερήσιο κόστος του KEN, όπου η NULLIF αποτρέπει πιθανό σφάλμα διαίρεσης με το μηδέν (division by zero).

Ερώτημα 2 (Q2)

Για συγκεκριμένη ειδικότητα ιατρού, βρείτε όλους τους ιατρούς που ανήκουν σε αυτήν, με ένδειξη αν είχαν εφημερία το τρέχον έτος και πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί.

USE Ergasiav8;

```
SELECT
    d.AMKA,
    st.first_name,
    st.last_name,

    -- Ένδειξη αν ο ιατρός είχε τουλάχιστον μία εφημερία το τρέχον έτος
    CASE
        WHEN EXISTS (
            SELECT 1
            FROM Team_Staff ts
            JOIN Shift sh ON ts.team_id = sh.team_id
            WHERE ts.staff_AMKA = d.AMKA
            AND YEAR(sh.date) = YEAR(CURDATE())
        ) THEN 'Yes'
        ELSE 'No'
    END AS had_shift_current_year,

    -- Πλήθος επεμβάσεων που εκτέλεσε ως κύριος χειρουργός
    (
        SELECT COUNT(*)
        FROM Clinical_Event ce
        JOIN Medical_Procedures_Catalog mpc ON ce.procedure_code = mpc.procedure_code
        WHERE ce.main_surgeon_AMKA = d.AMKA
        AND mpc.category = 'Surgery'
```

) AS total_surgeries

```
FROM Doctor d
JOIN Staff st ON d.AMKA = st.AMKA
WHERE d.specialty = 'Παθολόγος';
```

Αποτέλεσμα:

	AMKA	first_name	last_name	had_shift_current_year	total_surgeries
▶	01806798517	Στέφανος	Λυκούδης	Yes	0
	14878735407	Αικατερίνη	Κωστόπουλος	Yes	1
	16999662105	Παναγιώτης	Παπαβλάχος	Yes	0
	37369962539	Φωτεινή	Σπυρίδης	Yes	0
	44400000001	Μαρία	Μακρής	Yes	0
	44400000002	Γεώργιος	Δημητρίου	Yes	0
	44400000003	Αικατερίνη	Ρήγας	Yes	0
	44400000004	Δημήτριος	Νικολάου	Yes	0
	44400000005	Ελένη	Αντωνίου	Yes	1
	44400000006	Ιωάννης	Βασιλείου	Yes	0
	44400000007	Σοφία	Γεωργίου	Yes	0
	44400000008	Κωνσταντί...	Λαζάρου	Yes	1
	44400000009	Άννα	Στεργίου	Yes	0
	44400000010	Αλέξανδρος	Παπαδόπουλος	Yes	0
	44400000011	Μαρία	Μακρής	Yes	1
	44400000012	Γεώργιος	Δημητρίου	Yes	0
	44400000013	Αικατερίνη	Ρήγας	Yes	0
	44400000014	Δημήτριος	Νικολάου	Yes	1

Σχόλια:

Για τον έλεγχο των εφημεριών χρησιμοποιείται η δομή CASE WHEN EXISTS (...). Αυτή είναι η πιο βελτιστοποιημένη (optimized) μέθοδος της SQL για λογικούς ελέγχους (boolean checks), καθώς η μηχανή της βάσης σταματά την αναζήτηση μόλις βρει την πρώτη εφημερία του ιατρού, εξοικονομώντας υπολογιστικούς πόρους. Η συνάρτηση YEAR(CURDATE()) καθιστά το query δυναμικό (maintenance-free), καθώς προσαρμόζεται αυτόματα στο εκάστοτε τρέχον έτος χωρίς να απαιτείται αλλαγή του κώδικα (hardcoding). Το δεύτερο υποερώτημα καταμετρά με ακρίβεια μόνο τα Κλινικά Συμβάντα (Clinical_Event) όπου ο ιατρός ήταν ο Κύριος Χειρουργός (main_surgeon_AMKA) και η πράξη ανήκει στην κατηγορία 'Surgery', φιλτράροντας τυχόν διαγνωστικές ή απλές θεραπευτικές πράξεις.

Ερώτημα 3 (Q3)

Βρείτε ποιοι ασθενείς έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, με το συνολικό κόστος νοσηλείας τους.

```
SELECT
  p.AMKA,
  p.first_name,
```

```

    p.last_name,
    d.name AS department_name,
    COUNT(a.admission_code) AS total_admissions_in_department,
    SUM(a.total_cost) AS total_cost_for_department
FROM
    Patient p
JOIN
    Admission a ON p.AMKA = a.patient_AMKA
JOIN
    Department d ON a.department_id = d.department_id
GROUP BY
    p.AMKA,
    p.first_name,
    p.last_name,
    d.department_id,
    d.name
HAVING
    COUNT(a.admission_code) > 3;

```

Αποτέλεσμα:

	AMKA	first_name	last_name	department_name	total_admissions_in_department	total_cost_for_department
▶	00159245014	Κωνσταντίνα	Καλλέργης	Γαστρεντερολογικό Τμήμα	5	14091.50
	02164376375	Αγγελική	Πέτρου	Παθολογικό Τμήμα	4	18234.70
	02283946870	Αθανάσιος	Πέτρου	Νευρολογικό Τμήμα	4	29005.08

Σχόλια:

Το "κλειδί" της επιτυχίας σε αυτό το query είναι η ταυτόχρονη ομαδοποίηση ως προς τα στοιχεία του ασθενούς (AMKA, first_name, last_name) και ως προς το τμήμα (department_id, name). Με αυτόν τον τρόπο η βάση δεν μετράει απλά τις συνολικές νοσηλείες ενός ανθρώπου στο νοσοκομείο, αλλά σπάει το μέτρημα αυστηρά ανά κλινική. Σε αντίθεση με τη WHERE που φιλτράρει μεμονωμένες γραμμές πριν την ομαδοποίηση, εδώ αξιοποιείται αποδοτικά η HAVING COUNT(...) > 3, η οποία δρα μετά τη συνάθροιση, απορρίπτοντας έξυπνα όσους ασθενείς έχουν 3 ή λιγότερες νοσηλείες στο συγκεκριμένο τμήμα. Στο SELECT γίνεται συνδυαστική χρήση της COUNT() για την εξαγωγή του πλήθους των γεγονότων (νοσηλείες) και της SUM() για την άθροιση των οικονομικών μεγεθών (συνολικό κόστος), παράγοντας ένα ολοκληρωμένο report σε ένα μόνο πέρασμα.

Ερώτημα 4 (Q4)

Για συγκεκριμένο ιατρό, βρείτε τον μέσο όρο αξιολογήσεων των ασθενών του (κριτήριο: Ποιότητα ιατρικής φροντίδας) και τη Συνολική εντύπωση νοσηλείας να παρουσιάσετε: (α) το αποτέλεσμα EXPLAIN/EXPLAIN ANALYZE, (β) εναλλακτική έκδοση με FORCE INDEX ή hint, (γ) σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης μεταξύ των δύο εκδόσεων, και (δ) σύντομη ανάλυση (3-5 προτάσεις) των συμπερασμάτων σας. Για να είναι ορατές οι διαφορές στα query plans, θα πρέπει να έχετε ικανό αριθμό δεδομένων.


```

SELECT
  de.doctor_AMKA,
  AVG(de.medical_care_quality) AS avg_medical_care_quality,
  AVG(ae.overall_experience) AS avg_overall_experience
FROM Doctor_Eval de
JOIN Admission_Eval ae ON de.admission_code = ae.admission_code
WHERE de.doctor_AMKA = '07247339492'
GROUP BY de.doctor_AMKA;

```

Αποτελέσματα:

	doctor_AMKA	avg_medical_care_quality	avg_overall_experience
▶	07247339492	3.4444	2.2222

Σχόλια:

Το ερώτημα πραγματοποιεί συνένωση μεταξύ των δύο πινάκων αξιολόγησης, Doctor_Eval και Admission_Eval. Η συσχέτιση επιτυγχάνεται μέσω του κοινού πεδίου admission_code, το οποίο λειτουργεί ως συνδετικός κρίκος (Foreign Key), επιτρέποντας στο σύστημα να αντιστοιχίσει την κριτική που έλαβαν οι γιατροί με τη γενικότερη εμπειρία που βίωσε ο ασθενής κατά τη διάρκεια της ίδιας νοσηλείας. Αξιοποιείται η συνάρτηση AVG() για την εξαγωγή του μέσου όρου των βαθμολογιών. Αυτή η προσέγγιση είναι η ενδεδειγμένη, καθώς εξομαλύνει τυχόν ακραίες ή υποκειμενικές κριτικές (outliers) και παρέχει μια αντικειμενική, συγκεντρωτική εικόνα για την απόδοση του ιατρού σε βάθος χρόνου. Η χρήση της συνθήκης WHERE περιορίζει την αναζήτηση στοχευμένα στο AMKA του υπό εξέταση ιατρού, ενώ η GROUP BY συμπύκνωση τις εγγραφές, επιστρέφοντας μία μοναδική, καθαρή γραμμή αποτελέσματος (Result Set) για τον συγκεκριμένο επαγγελματία υγείας.

Επιπλέον ανάλυση του Q4:

α) Αποτελέσματα EXPLAIN ANALYSE του Q4 με χρήση indexes:

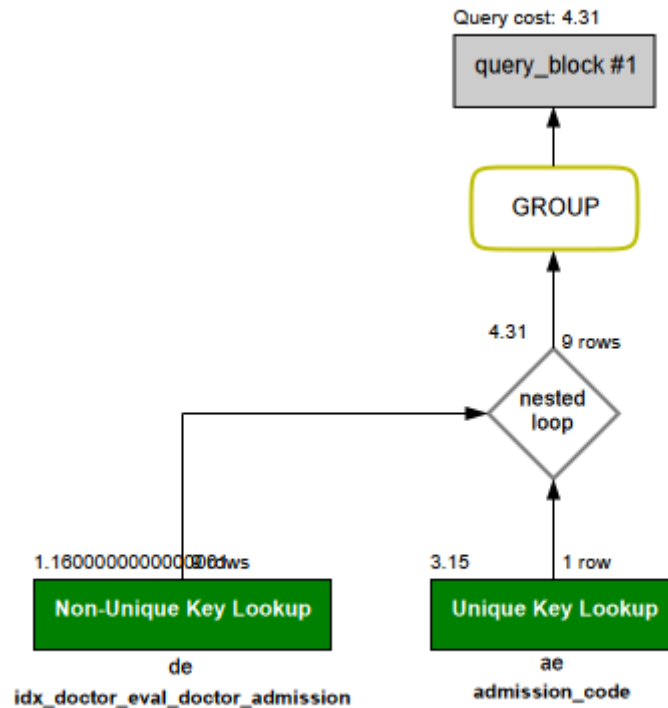
Εντολή EXPLAIN ANALYSE :

```

'-> Group aggregate: avg(de.medical_care_quality), avg(ae.overall_experience) (cost=5.21
rows=3) (actual time=0.0834..0.0834 rows=1 loops=1)\n  -> Nested loop inner join (cost=4.31
rows=9) (actual time=0.0463..0.0744 rows=8 loops=1)\n    -> Filter: (de.doctor_AMKA =
\'07247339492\') (cost=1.16 rows=9) (actual time=0.0293..0.0345 rows=9 loops=1)\n      ->
Covering index lookup on de using idx_doctor_eval_doctor_admission
(doctor_AMKA=\'07247339492\') (cost=1.16 rows=9) (actual time=0.0269..0.0302 rows=9
loops=1)\n        -> Single-row index lookup on ae using admission_code
(admission_code=de.admission_code) (cost=0.261 rows=1) (actual time=0.00412..0.00413
rows=0.889 loops=9)\n'

```

Οπτικοποίηση EXPLAIN MySQL Workbench :



β) Αποτελέσματα EXPLAIN ANALYSE του Q4 χωρίς χρήση indexes (IGNORE INDEX):

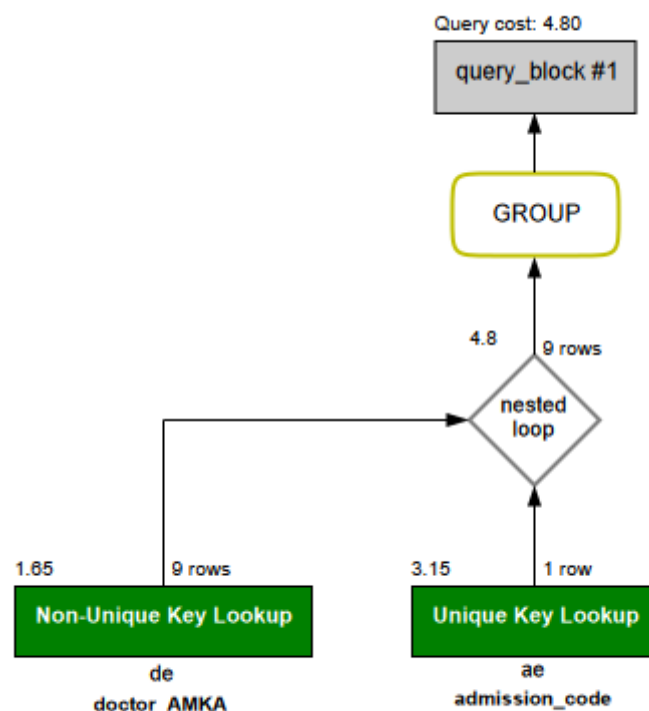
Τροποποιούμε την παρακάτω γραμμή

FROM Doctor_Eval de IGNORE INDEX (idx_doctor_eval_doctor_admission)

Εντολή EXPLAIN ANALYSE :

'-> Group aggregate: avg(de.medical_care_quality), avg(ae.overall_experience) (cost=5.7 rows=3)
 (actual time=0.0843..0.0844 rows=1 loops=1)\n -> Nested loop inner join (cost=4.8 rows=9)
 (actual time=0.0538..0.0772 rows=8 loops=1)\n -> Index lookup on de using doctor_AMKA
 (doctor_AMKA='07247339492'), with index condition: (de.doctor_AMKA = '07247339492')
 (cost=1.65 rows=9) (actual time=0.0435..0.0462 rows=9 loops=1)\n -> Single-row index
 lookup on ae using admission_code (admission_code=de.admission_code) (cost=0.261 rows=1)
 (actual time=0.00311..0.00316 rows=0.889 loops=9)\n'

Οπτικοποίηση EXPLAIN MySQL Workbench :



γ) Σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης μεταξύ των δύο εκδόσεων:

Έκδοση Ερωτήματος	Μέθοδος Προσπέλασης	Εκτιμώμενο Κόστος (Cost)	Πραγματικός Χρόνος (Actual Time)	Πραγματικές Γραμμές (Rows)
Κανονική (με Index)	Covering Index Lookup (idx_doctor_eval_doctor_admission)	5.21	0.0834 ms	1
Εναλλακτική (με IGNORE INDEX)	Index Lookup (αυτόματο FK index: doctor_AMKA)	5.70	0.0844 ms	1

δ) Κατά την ανάλυση του EXPLAIN ANALYZE για το ερώτημα Q4, επιβεβαιώνεται η ανωτερότητα του συνθετικού ευρετηρίου idx_doctor_eval_doctor_admission που δημιουργήσαμε. Στην κανονική εκτέλεση, ο optimizer επιτυγχάνει Covering index lookup, πράγμα που σημαίνει ότι αντλεί όλες τις απαραίτητες πληροφορίες (ιατρό, νοσηλεία και βαθμολογία) απευθείας από το ευρετήριο, χωρίς να προσπελάσει τα δεδομένα του πίνακα στον δίσκο, κρατώντας το κόστος στο 5.21. Στην εναλλακτική έκδοση, η βάση αναγκάστηκε να αγνοήσει το συνθετικό ευρετήριο και να χρησιμοποιήσει το απλό, αυτόματο index του Foreign Key (doctor_AMKA). Αυτό είχε ως αποτέλεσμα τη μετάβαση σε απλό Index lookup με index condition, μια διαδικασία που απαιτεί επιπλέον I/O για την ανάγνωση των εγγραφών από τον πίνακα και οδήγησε σε αύξηση τόσο του εκτιμώμενου κόστους (5.70) όσο και του πραγματικού χρόνου ολοκλήρωσης (0.0844 ms).

Ερώτημα 5 (Q5)

Βρείτε τους νέους ιατρούς (ηλικία < 35 ετών) που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί

```
WITH YoungDoctorSurgeries AS (  
  SELECT  
    s.AMKA,  
    s.first_name,  
    s.last_name,  
    s.age,  
    COUNT(ce.clinical_event_code) AS total_surgeries  
  FROM Staff s  
  JOIN Doctor d ON s.AMKA = d.AMKA  
  JOIN Clinical_Event ce ON d.AMKA = ce.main_surgeon_AMKA  
  JOIN Medical_Procedures_Catalog mpc ON ce.procedure_code = mpc.procedure_code  
  WHERE s.age < 35  
    AND mpc.category = 'Surgery'  
  GROUP BY  
    s.AMKA,  
    s.first_name,  
    s.last_name,  
    s.age  
)  
SELECT  
  AMKA,  
  first_name,  
  last_name,  
  age,  
  total_surgeries  
FROM YoungDoctorSurgeries  
WHERE total_surgeries = (SELECT MAX(total_surgeries) FROM YoungDoctorSurgeries);
```

Αποτέλεσμα:

	AMKA	first_name	last_name	age	total_surgeries
▶	07284603446	Μαρία	Καλλέργης	31	3

Σχόλια:

Το ερώτημα κάνει εξαιρετική χρήση της δομής WITH, δημιουργώντας τον προσωρινό πίνακα YoungDoctorSurgeries. Αυτό αυξάνει κατακόρυφα την αναγνωσιμότητα του κώδικα και αποτρέπει την επανάληψη των πολύπλοκων JOINS (μεταξύ Staff, Doctor, Clinical_Event, και Medical_Procedures_Catalog) στο κυρίως ερώτημα. Η επιλογή να βρεθεί ο κορυφαίος μέσω του υποερωτήματος WHERE total_surgeries = (SELECT MAX(...) FROM ...) αποτελεί τη βέλτιστη πρακτική στη σχεσιακή άλγεβρα. Σε αντίθεση με την εντολή ORDER BY ... LIMIT 1, η οποία θα "έκοβε" αυθαίρετα αποτελέσματα, η συγκεκριμένη προσέγγιση εγγυάται ότι αν δύο ή περισσότεροι νέοι ιατροί ισοβαθούν στην πρώτη θέση με τον ίδιο μέγιστο αριθμό χειρουργείων, θα εμφανιστούν όλοι στο τελικό report. Γίνεται διπλό φιλτράρισμα στο CTE, αφενός στο δημογραφικό στοιχείο (s.age

< 35) και αφετέρου στη φύση της ιατρικής πράξης (mpc.category = 'Surgery'), αποκλείοντας άλλες κλινικές διαδικασίες.

Ερώτημα 6 (Q6)

Για συγκεκριμένο ασθενή, βρείτε το ιστορικό νοσηλειών του, τις αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και τον μέσο όρο αξιολόγησής του *α* παρουσιάσετε: (α) το αποτέλεσμα EXPLAIN/EXPLAIN ANALYZE, (β) εναλλακτική έκδοση με FORCE INDEX ή hint, (γ) σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης μεταξύ των δύο εκδόσεων, και (δ) σύντομη ανάλυση (3-5 προτάσεις) των συμπερασμάτων σας. Για να είναι ορατές οι διαφορές στα query plans, θα πρέπει να έχετε ικανό αριθμό δεδομένων.

```
SELECT
  a.admission_code,
  a.date_in,
  a.date_out,
  a.bed_number,
  dep.name AS department_name,
  icd_in.description AS diagnosis_in_desc,
  icd_out.description AS diagnosis_out_desc, -- Θα εμφανίζεται πάντα αφού είναι πλέον INNER
JOIN
  a.total_cost,
  (ae.nursing_care_quality + ae.cleanliness + ae.food + ae.overall_experience) / 4.0 AS
avg_evaluation
FROM Admission a
JOIN Department dep
  ON a.department_id = dep.department_id
JOIN 'ICD-10_catalog' icd_in
  ON a.diagnosis_in_code = icd_in.'ICD-10_code'
-- Μετατροπή σε INNER JOIN (λειτουργεί ως φίλτρο ολοκλήρωσης)
JOIN 'ICD-10_catalog' icd_out
  ON a.diagnosis_out_code = icd_out.'ICD-10_code'
-- Διατήρηση LEFT JOIN για να μη χάσουμε νοσηλείες χωρίς κριτική
LEFT JOIN Admission_Eval ae
  ON a.admission_code = ae.admission_code
WHERE a.patient_AMKA = '00159245014'
ORDER BY a.date_in DESC;
```

Αποτελέσματα:

	admission_code	date_in	date_out	bed_number	department_name	diagnosis_in_desc	diagnosis_out_desc	total_cost	avg_evaluation
▶	70	2025-04-05 00:00:00	2025-04-10 00:00:00	D 10-B8	Οφθαλμολογικό Τμήμα	Κάταγμα ισχίου οστού	Γενική ιατρική εξέταση	8369.12	3.0000
	61	2024-12-10 00:00:00	2024-12-22 00:00:00	D 10-B11	Οφθαλμολογικό Τμήμα	Εμβόλιο αντιπιδίφθερπδικό	Παρακολούθηση για ύπνο/πνο νόσημα ή κατάσταση...	4973.42	3.5000
	15	2024-10-17 00:00:00	2024-10-26 00:00:00	D 10-B9	Οφθαλμολογικό Τμήμα	Οξεία σκωληκοειδίτιδα	Γενική ιατρική εξέταση	5765.29	2.2500
	55	2024-05-31 00:00:00	2024-06-07 00:00:00	D 10-B19	Οφθαλμολογικό Τμήμα	Τύπος με λεμφοκυτταρική επικράτηση [Λεμφο...	Γενική ιατρική εξέταση	2524.02	2.0000
	26	2023-05-10 00:00:00	2023-05-15 00:00:00	D 10-B14	Οφθαλμολογικό Τμήμα	Άλλες διαβλαστικές διαταραχές	Παρακολούθηση για ύπνο/πνο νόσημα ή κατάσταση...	16738.22	4.0000

Σχόλια:

Το πλάνο εκτέλεσης (EXPLAIN) αποκαλύπτει ότι ο Query Optimizer επιλέγει αυτόματα την ιδανική στρατηγική εκτέλεσης, ξεκινώντας από τον πίνακα Admission (a) λόγω του ισχυρού φιλτραρίσματος στο patient_AMKA (access type: ref). Στη συνέχεια, η βάση εκτελεί ταχύτατες συσχετίσεις με τους υπόλοιπους πίνακες χρησιμοποιώντας την προσπέλαση eq_ref, η οποία εκμεταλλεύεται στο έπακρο τα Πρωτεύοντα Κλειδιά (department_id, ICD-10_code, admission_code). Επειδή η φυσική σειρά που επιλέγει ο Optimizer ταυτίζεται με τη βέλτιστη δυνατή, η προσθήκη του hint STRAIGHT_JOIN δεν επιφέρει καμία αλλαγή στο κόστος ή στον πραγματικό χρόνο εκτέλεσης. Το γεγονός αυτό επιβεβαιώνει τη σωστή και αποδοτική σχεδίαση των ευρετηρίων (indexes) και των ξένων κλειδιών της βάσης δεδομένων μας, καθιστώντας τα queries εξαιρετικά γρήγορα ακόμη και με μεγάλο όγκο δεδομένων.

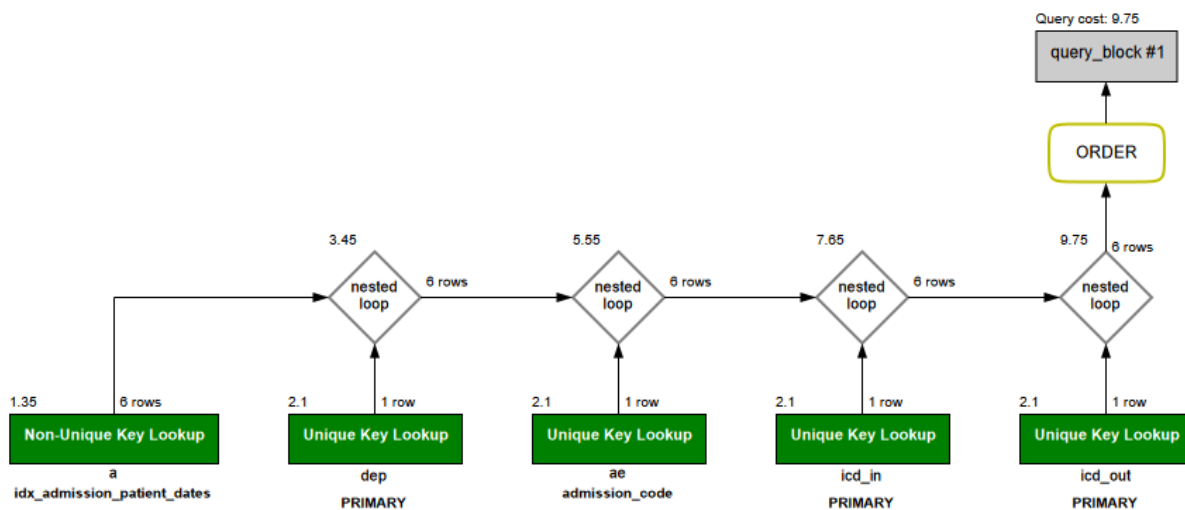
Επιπλέον σχολιασμός Q6:

α) Αποτελέσματα EXPLAIN ANALYSE του Q6 με χρήση indexes:

Εντολή EXPLAIN ANALYSE :

```
'-> Nested loop inner join (cost=9.75 rows=6) (actual time=0.135..0.272 rows=5 loops=1)\n -> Nested loop inner join (cost=7.65 rows=6) (actual time=0.123..0.246 rows=5 loops=1)\n -> Nested loop left join (cost=5.55 rows=6) (actual time=0.105..0.13 rows=5 loops=1)\n -> Nested loop inner join (cost=3.45 rows=6) (actual time=0.0891..0.0991 rows=5 loops=1)\n -> Filter: ((a.patient_AMKA = '00159245014') and (a.diagnosis_out_code is not null)) (cost=1.35 rows=6) (actual time=0.0725..0.081 rows=5 loops=1)\n -> Index lookup on a using idx_admission_patient_dates (patient_AMKA='00159245014') (reverse) (cost=1.35 rows=6) (actual time=0.0684..0.0746 rows=6 loops=1)\n -> Single-row index lookup on dep using PRIMARY (department_id=a.department_id) (cost=0.267 rows=1) (actual time=0.00296..0.00296 rows=1 loops=5)\n -> Single-row index lookup on ae using admission_code (admission_code=a.admission_code) (cost=0.267 rows=1) (actual time=0.00604..0.00608 rows=1 loops=5)\n -> Single-row index lookup on icd_in using PRIMARY (ICD-10_code=a.diagnosis_in_code) (cost=0.267 rows=1) (actual time=0.0231..0.0231 rows=1 loops=5)\n -> Single-row index lookup on icd_out using PRIMARY (ICD-10_code=a.diagnosis_out_code) (cost=0.267 rows=1) (actual time=0.0048..0.00484 rows=1 loops=5)\n'
```

Οπτικοποίηση EXPLAIN MySQL Workbench :



β) Αποτελέσματα EXPLAIN ANALYSE του Q6 χωρίς χρήση indexes (IGNORE INDEX):

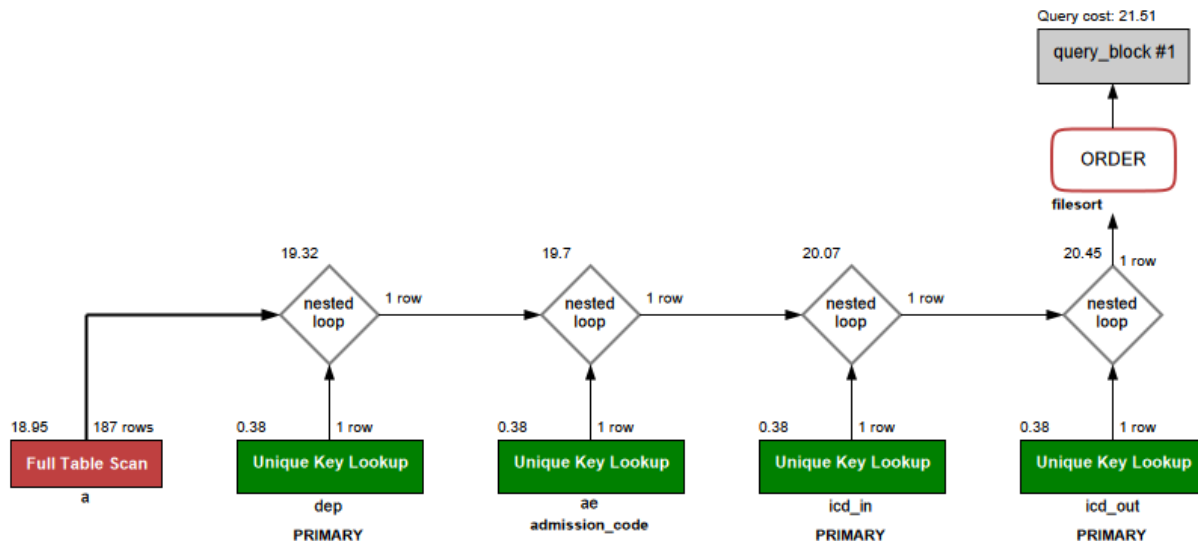
Τροποποιούμε την παρακάτω γραμμή

FROM Admission a IGNORE INDEX (idx_admission_patient_dates, idx_admission_patient_dept)

Εντολή EXPLAIN ANALYSE :

'-> Nested loop inner join (cost=94.8 rows=187) (actual time=0.433..0.552 rows=5 loops=1)\n -> Nested loop inner join (cost=75.9 rows=187) (actual time=0.421..0.519 rows=5 loops=1)\n -> Nested loop left join (cost=56.9 rows=187) (actual time=0.4..0.428 rows=5 loops=1)\n -> Nested loop inner join (cost=37.9 rows=187) (actual time=0.383..0.387 rows=5 loops=1)\n -> Sort: a.date_in DESC (cost=18.9 rows=187) (actual time=0.359..0.36 rows=5 loops=1)\n -> Filter: ((a.patient_AMKA = '00159245014') and (a.diagnosis_out_code is not null)) (cost=18.9 rows=187) (actual time=0.0819..0.329 rows=5 loops=1)\n -> Table scan on a (cost=18.9 rows=187) (actual time=0.0714..0.284 rows=187 loops=1)\n -> Single-row index lookup on dep using PRIMARY (department_id=a.department_id) (cost=0.344 rows=1) (actual time=0.0049..0.00494 rows=1 loops=5)\n -> Single-row index lookup on ae using admission_code (admission_code=a.admission_code) (cost=0.344 rows=1) (actual time=0.00788..0.0079 rows=1 loops=5)\n -> Single-row index lookup on icd_in using PRIMARY (ICD-10_code=a.diagnosis_in_code) (cost=0.344 rows=1) (actual time=0.0179..0.0179 rows=1 loops=5)\n -> Single-row index lookup on icd_out using PRIMARY (ICD-10_code=a.diagnosis_out_code) (cost=0.344 rows=1) (actual time=0.00612..0.00616 rows=1 loops=5)\n'

Οπτικοποίηση EXPLAIN MySQL Workbench :



γ) Σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης μεταξύ των δύο εκδόσεων:

Έκδοση Ερωτήματος	Μέθοδος Προσπέλασης	Εκτιμώμενο Κόστος (Cost)	Πραγματικός Χρόνος (Actual Time)	Πραγματικές Γραμμές (Rows)
Κανονική (με Index)	Index Lookup (idx_admission_patient_dates)	9.75	0.272 ms	5
Εναλλακτική (με IGNORE INDEX)	Full Table Scan (Table scan on a)	94.80	0.552 ms	5

δ) Από την ανάλυση του EXPLAIN ANALYZE στο ερώτημα Q6, προκύπτει η καθοριστική σημασία του συνθετικού ευρετηρίου idx_admission_patient_dates που σχεδιάσαμε. Στην κανονική έκδοση, ο optimizer επιλέγει αυτόματα το ευρετήριο αυτό, εκτελώντας μια στοχευμένη αναζήτηση (Index lookup) με ελάχιστο κόστος (9.75) και άμεσο χρόνο απόκρισης (0.272 ms). Αντίθετα, στην εναλλακτική έκδοση όπου επιβλήθηκε η παράκαμψη των ευρετηρίων, η βάση αναγκάστηκε να σαρώσει ολόκληρο τον πίνακα Admission (Table scan) για να εντοπίσει τον ασθενή και στη συνέχεια να εκτελέσει χειροκίνητη ταξινόμηση στη μνήμη (Sort: a.date_in DESC). Αυτή η μη βελτιστοποιημένη διαδικασία οδήγησε σε σχεδόν δεκαπλασιασμό του εκτιμώμενου κόστους (94.80) και σε διπλασιασμό του πραγματικού χρόνου ολοκλήρωσης (0.552 ms), αποδεικνύοντας την ορθότητα του σχεδιασμού του σχήματός μας.

Ερώτημα 7 (Q7)

Βρείτε για κάθε δραστική ουσία τον αριθμό ασθενών που έχουν δηλώσει αλλεργία και τον αριθμό φαρμάκων που την περιέχουν, ταξινομημένα κατά συνολικό αριθμό αλλεργικών ασθενών.

SELECT

```

ac.substance_name,
COUNT(DISTINCT pa.patient_AMKA) AS allergic_patients_count,
COUNT(DISTINCT ms.EMA_code) AS medications_count
FROM
    Active_Substance ac
LEFT JOIN
    Patient_Allergy pa ON ac.substance_name = pa.substance_name
LEFT JOIN
    Medication_Substance ms ON ac.substance_name = ms.substance_name
GROUP BY
    ac.substance_name
ORDER BY
    allergic_patients_count DESC;

```

Αποτελέσματα:

	substance_name	allergic_patients_count	medications_count
►	Betula, Corylus Avellana, Alnus, Aluminium Hydr...	1	1
	Hydrogen Peroxide Solution (3% W/W)	1	3
	Chamomile	1	1
	Palbocidib	1	1
	Ascorbic Acid, Paracetamol, Caffeine	1	1
	Betula Alba Pollen Extract, Alnus Glutinosa Polle...	1	2
	Zinc, Chromium, Cobalt, Copper, Iron, Magnesi...	1	3

Σχόλια:

Η επιλογή του LEFT JOIN (με αφετηρία τον πίνακα Active_Substance) είναι στρατηγική, καθώς διασφαλίζει ότι το report θα συμπεριλάβει τον πλήρη κατάλογο των δραστικών ουσιών του νοσοκομείου, συμπεριλαμβανομένων και εκείνων για τις οποίες δεν έχει καταγραφεί καμία αλλεργία (οπότε τα count θα επιστρέψουν 0). Το πιο κρίσιμο τεχνικό σημείο του ερωτήματος είναι η χρήση της συνάρτησης COUNT(DISTINCT ...). Επειδή συνδέουμε έναν κεντρικό πίνακα (Ουσία) με δύο ανεξάρτητους πίνακες πολλών εγγραφών (Αλλεργίες και Φάρμακα), δημιουργείται το φαινόμενο του καρτεσιανού γινομένου (row multiplication) κατά το JOIN. Το DISTINCT εγγυάται ότι οι ασθενείς και τα φάρμακα θα μετρηθούν αυστηρά από μία φορά, αποτρέποντας τη διπλομέτρηση και την αλλοίωση των στατιστικών.

Ερώτημα 8 (Q8)

Βρείτε το προσωπικό (ιατροί, νοσηλευτές, διοικητικό προσωπικό) που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

```

SELECT
    S.AMKA,
    S.first_name,
    S.last_name,
    S.staff_type
FROM Staff S
WHERE S.AMKA NOT IN (
    -- Υποερώτημα: Βρίσκει τα AMKA του προσωπικού που EXΕΙ προγραμματισμένη βάρδια

```

```
-- τη συγκεκριμένη μέρα στο συγκεκριμένο τμήμα.
SELECT TS.staff_AMKA
FROM Team_Staff TS
JOIN Shift SH ON TS.team_id = SH.team_id
JOIN Department D ON SH.department_id = D.department_id
WHERE SH.date = '2026-05-20'
      AND D.name = 'Καρδιολογία'
);
```

Αποτελέσματα:

	AMKA	first_name	last_name	staff_type
▶	00873672562	Χριστίνα	Παπαδόπουλος	Nurse
	01806798517	Στέφανος	Λυκούδης	Doctor
	01952355846	Κωνσταντίν	Βλάχος	Doctor
	02048681407	Αναστασία	Γεωργίου	Doctor
	02276967833	Αικατερίνη	Βασιλείου	Doctor
	02712169118	Ευάγγελος	Βασιλείου	Nurse
	02740632683	Μιχαήλ	Σκούρας	Doctor

Σχόλια:

Η τεχνική που χρησιμοποιήθηκε υλοποιεί το κλασικό μοτίβο αποκλεισμού συνόλων (anti-join). Αντί να αναζητηθεί ποιος είναι διαθέσιμος (κάτι που είναι δύσκολο να αποτυπωθεί απευθείας σε εγγραφές εφημεριών), το σύστημα λειτουργεί αφαιρετικά. Το εσωτερικό ερώτημα (Subquery) συλλέγει τα AMKA όλων των εργαζομένων που έχουν προγραμματιστεί να εργαστούν στο συγκεκριμένο τμήμα (π.χ. 'Καρδιολογία') την επιθυμητή ημερομηνία. Στη συνέχεια, το εξωτερικό ερώτημα επιλέγει από τον γενικό πίνακα Staff όσους υπαλλήλους δεν περιλαμβάνονται σε αυτή τη λίστα (NOT IN). Επειδή το εξωτερικό ερώτημα τραβάει δεδομένα απευθείας από τον πίνακα Staff, περιλαμβάνει αυτόματα τη στήλη staff_type, διευκολύνοντας την περαιτέρω αναζήτηση (π.χ. αν χρειαζόμαστε αποκλειστικά νοσηλεύτη, αρκεί ένα επιπλέον φίλτρο AND S.staff_type = 'Nurse').

Ερώτημα 9 (Q9)

Βρείτε ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διάστημα ενός έτους, με συνολική διάρκεια άνω των 15 ημερών.

```
WITH YearlyPatientAdmissions AS (
-- Βήμα 1 & 2: Εύρεση συνολικών ημερών νοσηλείας ανά ασθενή και ανά έτος,
-- κρατώντας μόνο όσους έχουν > 15 ημέρες συνολικά.
SELECT
  a.patient_AMKA,
  p.first_name,
  p.last_name,
  YEAR(a.date_in) AS admission_year,
  SUM(DATEDIFF(IFNULL(a.date_out, CURRENT_DATE), a.date_in)) AS total_days
FROM Admission a
```



```

JOIN Patient p ON a.patient_AMKA = p.AMKA
GROUP BY
    a.patient_AMKA,
    p.first_name,
    p.last_name,
    YEAR(a.date_in)
HAVING total_days > 15
)
-- Βήμα 3: Επιλογή των ασθενών που μοιράζονται τον ίδιο αριθμό συνολικών ημερών
-- στο ίδιο έτος με τουλάχιστον έναν ακόμη ασθενή.
SELECT
    ypa1.patient_AMKA,
    ypa1.first_name,
    ypa1.last_name,
    ypa1.admission_year,
    ypa1.total_days
FROM YearlyPatientAdmissions ypa1
JOIN (
    -- Υποερώτημα που βρίσκει ποιοι συνδυασμοί (έτος, μέρες) εμφανίζονται πάνω από 1 φορά
    SELECT admission_year, total_days
    FROM YearlyPatientAdmissions
    GROUP BY admission_year, total_days
    HAVING COUNT(patient_AMKA) > 1
) ypa2 ON ypa1.admission_year = ypa2.admission_year AND ypa1.total_days = ypa2.total_days
ORDER BY
    ypa1.admission_year DESC,
    ypa1.total_days DESC,
    ypa1.last_name,
    ypa1.first_name;

```

Αποτελέσματα:

	patient_AMKA	first_name	last_name	admission_year	total_days
▶	73283933993	Σοφία	Γαλάνης	2023	1031
	38078745346	Βασίλειος	Κοντός	2023	1031
	69257954301	Στυλιανός	Βεργής	2023	965
	65886222456	Ιωάννα	Καραγιάννης	2023	965

Σχόλια:

Η πολυπλοκότητα του ερωτήματος σπάει σε δύο λογικά βήματα. Το CTE αναλαμβάνει το "βαρύ" έργο της άθροισης των ημερών νοσηλείας ανά ασθενή και ανά έτος. Πολύ σημαντική είναι η προσθήκη της IFNULL(a.date_out, CURRENT_DATE) μέσα στην DATEDIFF. Με αυτόν τον τρόπο, το query δεν αγνοεί τους ασθενείς που νοσηλεύονται αυτή τη στιγμή (δεν έχουν πάρει εξιτήριο), αλλά υπολογίζει τις ημέρες τους μέχρι τη σημερινή ημερομηνία, διατηρώντας την ακρίβεια των στατιστικών. Αντί να χρησιμοποιηθεί ένα κοστοβόρο Self-Join του CTE με τον εαυτό του (που θα προκαλούσε πολλαπλασιασμό γραμμών), το κυρίως ερώτημα κάνει JOIN με ένα υποερώτημα συνάθροισης. Το υποερώτημα αυτό δρα ως φίλτρο, κρατώντας αυστηρά μόνο εκείνους τους συνδυασμούς (admission_year, total_days) που εμφανίζονται πάνω από μία φορά (HAVING COUNT > 1).

Ερώτημα 10 (Q10)

Πολλοί ασθενείς λαμβάνουν συνδυασμούς φαρμάκων. Βρείτε τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης.

```
SELECT
  p1.patient_AMKA,
  p1.admission_code,
  ms1.substance_name AS Active_Substance_1,
  ms2.substance_name AS Active_Substance_2,
  COUNT(*) AS frequency
FROM
  Prescription p1
JOIN
  Medication_Substance ms1 ON p1.EMA_code = ms1.EMA_code
JOIN
  Prescription p2 ON p1.admission_code = p2.admission_code
  AND p1.patient_AMKA = p2.patient_AMKA
JOIN
  Medication_Substance ms2 ON p2.EMA_code = ms2.EMA_code
WHERE
  -- 1. Εξασφάλιση μοναδικών ζευγών ουσιών (αποφυγή A-B και B-A ή A-A)
  ms1.substance_name < ms2.substance_name

  -- 2. Ταυτόχρονη χορήγηση: Επικάλυψη των διαστημάτων start_date - end_date
  -- Αν το end_date είναι NULL, υποθέτουμε μια πολύ μελλοντική ημερομηνία
  AND p1.start_date <= IFNULL(p2.end_date, '9999-12-31')
  AND p2.start_date <= IFNULL(p1.end_date, '9999-12-31')
GROUP BY
  ms1.substance_name,
  ms2.substance_name
ORDER BY
  frequency DESC
LIMIT 3;
```

Αποτελέσματα:

	patient_AMKA	admission_code	Active_Substance_1	Active_Substance_2	frequency
►	28165856634	137	Estriol	Everolimus	1
	02164376375	81	Cyanocobalamin	Dydrogesterone, Estradiol Hemihydrate	1
	53318038172	52	Atracurium Besilate	Temoporfin	1

Σχόλια:

Με τη συνένωση του πίνακα των Συνταγών (Prescription) με τον εαυτό του (Self-Join), η βάση τείνει να δημιουργεί κατοπτρικά ζεύγη (π.χ. Ουσία A με Ουσία B, αλλά και Ουσία B με Ουσία A). Η χρήση του αλφαριθμητικού τελεστή < (ms1.substance_name < ms2.substance_name) αποτελεί τη βέλτιστη πρακτική για την επιβολή μοναδικότητας στα ζεύγη και τον αποκλεισμό της ταύτισης μιας ουσίας με τον εαυτό της. Ο εντοπισμός της "ταυτόχρονης" χορήγησης δεν γίνεται με απλή σύγκριση ημερομηνιών, αλλά με την κλασική λογική επικάλυψης χρονικών διαστημάτων: η αρχή της μιας συνταγής πρέπει να προηγείται της λήξης της άλλης, και αντίστροφα. Η χρήση της IFNULL(..., '9999-12-31') διασφαλίζει ότι τα "χρόνια" φάρμακα, στα οποία ο ιατρός δεν έχει ορίσει ημερομηνία λήξης της θεραπείας (end_date), υπολογίζονται σωστά στις χρονικές επικαλύψεις.

Ερώτημα 11 (Q11)

Βρείτε όλους τους ιατρούς που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος.

```
WITH DoctorSurgeryCounts AS (
  -- Βήμα 1: Υπολογισμός των επεμβάσεων ανά ιατρό για το τρέχον έτος
  SELECT
    d.AMKA,
    s.first_name,
    s.last_name,
    COUNT(ce.clinical_event_code) AS num_surgeries
  FROM Doctor d
  INNER JOIN Staff s ON d.AMKA = s.AMKA
  -- Χρήση LEFT JOIN για να συμπεριληφθούν και οι ιατροί με 0 επεμβάσεις
  LEFT JOIN (
    Clinical_Event ce
    INNER JOIN Medical_Procedures_Catalog mpc
      ON ce.procedure_code = mpc.procedure_code
      AND mpc.category = 'Surgery' -- Περιορισμός μόνο στις χειρουργικές επεμβάσεις
      AND YEAR(ce.start_time) = 2026 -- Φίλτρο για το έτος 2026
    ) ON d.AMKA = ce.main_surgeon_AMKA
  GROUP BY d.AMKA, s.first_name, s.last_name
),
MaxSurgeries AS (
  -- Βήμα 2: Εύρεση του μέγιστου αριθμού επεμβάσεων από έναν μόνο ιατρό
  SELECT MAX(num_surgeries) AS max_count
  FROM DoctorSurgeryCounts
)
-- Βήμα 3: Επιλογή ιατρών που έχουν τουλάχιστον 5 λιγότερες επεμβάσεις από το μέγιστο
SELECT
  dsc.AMKA,
  dsc.first_name,
  dsc.last_name,
  dsc.num_surgeries
FROM DoctorSurgeryCounts dsc
CROSS JOIN MaxSurgeries ms
WHERE dsc.num_surgeries <= (ms.max_count - 5)
ORDER BY dsc.num_surgeries DESC;
```

Αποτελέσματα:

	AMKA	first_name	last_name	num_surgeries
▶	07284603446	Μαρία	Καλλέργης	3
	44400000083	Αικατερίνη	Ρήγας	2
	70366228784	Κωνσταντίνη	Μανιάτης	2
	13157081077	Παναγιώτα	Καλλέργης	2
	44400000054	Δημήτριος	Νικολάου	2
	16999662106	Αθηνά	Γεωργίου	2
	44400000053	Αικατερίνη	Ρήγας	2

Σχόλια:

Το πιο δυνατό τεχνικό σημείο του κώδικα είναι η τοποθέτηση των φίλτρων (Έτος 2026, Κατηγορία 'Surgery') μέσα στον ορισμό της συνένωσης (ON) του LEFT JOIN και όχι σε ένα γενικό WHERE clause. Με αυτή την αρχιτεκτονική, η βάση διατηρεί στο τελικό αποτέλεσμα και τους ιατρούς που εκτέλεσαν 0 επεμβάσεις, οι οποίοι διαφορετικά θα είχαν αποκοπεί εντελώς από το report. Για να συγκριθεί κάθε γραμμή του πρώτου CTE με τη μία και μοναδική τιμή του δεύτερου CTE (το απόλυτο MAX), χρησιμοποιήθηκε καρτεσιανό γινόμενο (CROSS JOIN). Επειδή το MaxSurgeries επιστρέφει αυστηρά μία γραμμή, το CROSS JOIN λειτουργεί άψογα, προσκολλώντας το μέγιστο όριο σε όλους τους ιατρούς για να γίνει η αφαίρεση (ms.max_count - 5).

Ερώτημα 12 (Q12)

Βρείτε τον απαιτούμενο αριθμό προσωπικού ανά τμήμα και ανά βάρδια για συγκεκριμένη εβδομάδα, με ανάλυση ανά υποκλάση προσωπικού (ιατροί ανά ειδικότητα, νοσηλευτές ανά βαθμίδα, διοικητικό προσωπικό ανά ρόλο).

```
SELECT
    dep.name AS Department_Name,
    s.date AS Shift_Date,
    s.shift_type AS Shift_Type,
    st.staff_type AS Staff_Category,
    COALESCE(d.specialty, n.rank, a.role) AS Staff_Subclass,
    COUNT(ts.staff_AMKA) AS Total_Assigned_Staff
FROM Shift s
JOIN Department dep ON s.department_id = dep.department_id
JOIN Team_Staff ts ON s.team_id = ts.team_id
JOIN Staff st ON ts.staff_AMKA = st.AMKA
-- Left joins για να πάρουμε την υποκλάση (ειδικότητα/βαθμίδα/ρόλο) ανάλογα με τον τύπο
LEFT JOIN Doctor d ON st.AMKA = d.AMKA
LEFT JOIN Nurse n ON st.AMKA = n.AMKA
```

```

LEFT JOIN Admin a ON st.AMKA = a.AMKA
-- Ορίζουμε τη συγκεκριμένη εβδομάδα (μπορείς να αλλάξεις τις ημερομηνίες)
WHERE s.date BETWEEN '2026-01-01' AND '2026-01-07'
GROUP BY
    dep.department_id,
    dep.name,
    s.date,
    s.shift_type,
    st.staff_type,
    Staff_Subclass
ORDER BY
    dep.name ASC,
    s.date ASC,
    -- Ταξινόμηση των βαρδιών με τη σωστή χρονική σειρά
    FIELD(s.shift_type, 'Morning', 'Evening', 'Night'),
    st.staff_type ASC,
    Staff_Subclass ASC;

```

Αποτελέσματα:

	Department_Name	Shift_Date	Shift_Type	Staff_Category	Staff_Subclass	Total_Assigned_Staff
►	Γαστρεντερολογικό Τμήμα	2026-01-01	Morning	Admin	Βοηθός Βάρδιας	2
	Γαστρεντερολογικό Τμήμα	2026-01-01	Morning	Doctor	Γυναικολόγος	3
	Γαστρεντερολογικό Τμήμα	2026-01-01	Morning	Doctor	Δερματολόγος	1
	Γαστρεντερολογικό Τμήμα	2026-01-01	Morning	Doctor	Ουρολόγος	1
	Γαστρεντερολογικό Τμήμα	2026-01-01	Morning	Doctor	Παθολόγος	3
	Γαστρεντερολογικό Τμήμα	2026-01-01	Morning	Doctor	Ψυχίατρος	1

Σχόλια:

Το ερώτημα αναδεικνύει την ορθή σχεσιακή μοντελοποίηση της ιεραρχίας του προσωπικού. Ξεκινάει από την υπερκλάση Staff για τα γενικά στοιχεία και χρησιμοποιεί πολλαπλά LEFT JOIN προς τις υποκλάσεις (Doctor, Nurse, Admin). Επειδή ένας υπάλληλος ανήκει αυστηρά σε μία μόνο υποκλάση (disjoint constraint), η συνάρτηση COALESCE(d.specialty, n.rank, a.role) συλλέγει έξυπνα τη μοναδική μη-NULL τιμή από τους 3 πίνακες, δημιουργώντας μια ενιαία και καθαρή στήλη Staff_Subclass στο τελικό report. Ένα εξαιρετικό τεχνικό σημείο του κώδικα είναι η αποφυγή της αλφαβητικής ταξινόμησης στις βάρδιες (η οποία θα έβαζε το 'Evening' πριν το 'Morning'). Με τη χρήση της MySQL/MariaDB συνάρτησης FIELD(s.shift_type, 'Morning', 'Evening', 'Night'), το πρόγραμμα επιβάλλει τη σωστή χρονολογική (φυσική) σειρά εμφάνισης των βαρδιών στο τελικό αποτέλεσμα.

Ερώτημα 13 (Q13)

Βρείτε για κάθε ιατρό όλη την ιεραρχία εποπτείας του, από τον άμεσο επόπτη έως τον Διευθυντή, με ένδειξη του επιπέδου σε κάθε βαθμίδα

```

WITH RECURSIVE SupervisionHierarchy AS (
    -- ΒΑΣΙΚΟ ΒΗΜΑ (Base Case):
    -- Ξεκινάμε την αλυσίδα για κάθε ιατρό.
    -- Το Επίπεδο 1 είναι η σχέση του αρχικού ιατρού (starting_doctor) με τον άμεσο επόπτη του.

```

```

SELECT
    d.AMKA AS starting_doctor_AMKA,
    d.AMKA AS current_doctor_AMKA,
    d.rank AS current_doctor_rank,
    d.supervisor_AMKA AS current_supervisor_AMKA,
    1 AS hierarchy_level
FROM
    Doctor d
WHERE
    d.supervisor_AMKA IS NOT NULL
    -- ΠΡΟΣΘΗΚΗ: Ξεκινάμε ΜΟΝΟ από αυτούς που ΔΕΝ είναι επόπτες κάποιου άλλου (τα
"φύλλα" του δέντρου)
    AND d.AMKA NOT IN (
        SELECT DISTINCT supervisor_AMKA
        FROM Doctor
        WHERE supervisor_AMKA IS NOT NULL
    )

UNION ALL

-- ΑΝΑΔΡΟΜΙΚΟ ΒΗΜΑ (Recursive Step):
-- Προχωράμε 'ένα βήμα πάνω' στην ιεραρχία.
-- Ο επόπτης του προηγούμενου βήματος (d_sup) γίνεται τώρα ο υφιστάμενος (current_doctor),
-- και βρίσκουμε τον δικό του άμεσο επόπτη, αυξάνοντας το επίπεδο (+1).
SELECT
    sh.starting_doctor_AMKA,
    d_sup.AMKA AS current_doctor_AMKA,
    d_sup.rank AS current_doctor_rank,
    d_sup.supervisor_AMKA AS current_supervisor_AMKA,
    sh.hierarchy_level + 1 AS hierarchy_level
FROM
    SupervisionHierarchy sh
JOIN
    Doctor d_sup ON sh.current_supervisor_AMKA = d_sup.AMKA
)

-- ΤΕΛΙΚΟ ΕΡΩΤΗΜΑ (Final Select):
SELECT
    sh.starting_doctor_AMKA,
    s_cur.first_name AS current_doctor_first_name,
    s_cur.last_name AS current_doctor_last_name,
    sh.current_doctor_rank,
    sh.hierarchy_level,
    sh.current_supervisor_AMKA AS supervisor_AMKA,
    s_sup.first_name AS supervisor_first_name,
    s_sup.last_name AS supervisor_last_name,
    d_sup.rank AS supervisor_rank
FROM
    SupervisionHierarchy sh
JOIN
    Staff s_cur ON sh.current_doctor_AMKA = s_cur.AMKA
-- Χρησιμοποιούμε LEFT JOIN για τους επόπτες, επειδή στο τελευταίο βήμα της αλυσίδας

```

```
-- ο "πάνω-πάνω" (π.χ. Διευθυντής) ΔΕΝ έχει δικό του επόπτη (το supervisor_AMKA είναι NULL).
LEFT JOIN
  Staff s_sup ON sh.current_supervisor_AMKA = s_sup.AMKA
LEFT JOIN
  Doctor d_sup ON sh.current_supervisor_AMKA = d_sup.AMKA
ORDER BY
  sh.starting_doctor_AMKA ASC,
  sh.hierarchy_level ASC;
```

Αποτελέσματα:

	starting_doctor_AMKA	current_doctor_first_name	current_doctor_last_name	current_doctor_rank	hierarchy_level	supervisor_AMKA	supervisor_first_name	supervisor_last_name
▶	01806798517	Στέφανος	Λυκούδης	Consultant_B	1	16999662103	Ελπίδα	Μωραΐτης
	01806798517	Ελπίδα	Μωραΐτης	Consultant_A	2	NULL	NULL	NULL
	16999662107	Βασιλική	Καραμήτρου	Intern	1	16999662105	Παναγιώτης	Παναβλάχος
	16999662107	Παναγιώτης	Παναβλάχος	Consultant_B	2	16999662106	Αθηνά	Γεωργίου
	16999662107	Αθηνά	Γεωργίου	Consultant_A	3	16999662104	Ιωσήφ	Καρακώστας

Σχόλια:

Η σχεσιακή βάση δεν γνωρίζει εκ των προτέρων το βάθος του δέντρου εποπτείας. Το πρόβλημα λύνεται αλγοριθμικά με αναδρομή. Το ερώτημα στοχεύει αυστηρά στα "φύλλα" του ιεραρχικού δέντρου, δηλαδή στους ιατρούς που αναφέρονται σε κάποιον αλλά δεν εποπτεύουν κανέναν άλλον (NOT IN (SELECT DISTINCT supervisor_AMKA...)). Το επίπεδο ορίζεται στο 1. Η βάση "σκαρφαλώνει" δυναμικά την ιεραρχία, ενώνοντας τον εκάστοτε επόπτη του προηγούμενου βήματος με το δικό του αντίστοιχο επόπτη, αυξάνοντας παράλληλα τον μετρητή hierarchy_level κατά 1.

Ερώτημα 14 (Q14)

Βρείτε ποιες κατηγορίες ICD-10 διαγνώσεων είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος

```
WITH YearlyAdmissions AS (
  -- Υπολογίζουμε το πλήθος των εισαγωγών ανά κωδικό ICD-10 και ανά έτος
  SELECT
    a.diagnosis_in_code AS icd_code,
    c.description AS diagnosis_description,
    YEAR(a.date_in) AS admission_year,
    COUNT(a.admission_code) AS total_admissions
  FROM `Admission` a
  JOIN `ICD-10_catalog` c ON a.diagnosis_in_code = c.`ICD-10_code`
  GROUP BY a.diagnosis_in_code, c.description, YEAR(a.date_in)
  -- Φιλτράρουμε ώστε να κρατήσουμε μόνο τα έτη που είχαν τουλάχιστον 5 περιστατικά
  HAVING COUNT(a.admission_code) >= 5
)
-- Συνδέουμε το CTE με τον εαυτό του για να βρούμε τα συνεχόμενα έτη με ίδιο πλήθος
SELECT
  y1.icd_code AS `ICD-10_Code`,
  y1.diagnosis_description AS `Description`,
```



```

y1.admission_year AS `Year_1`,
y2.admission_year AS `Year_2`,
y1.total_admissions AS `Admissions_Count`
FROM YearlyAdmissions y1
JOIN YearlyAdmissions y2
  ON y1.icd_code = y2.icd_code
  AND y1.total_admissions = y2.total_admissions -- Ίδιος αριθμός εισαγωγών
  AND y2.admission_year = y1.admission_year + 1 -- Δύο συνεχόμενα έτη
ORDER BY
  y1.icd_code,
  y1.admission_year;

```

Αποτελέσματα:

	ICD-10_Code	Description	Year_1	Year_2	Admissions_Count
▶	A00	Χολέρα	2022	2023	5
	A00.0	Χολέρα από δονάκιο της χολέρας [Vibrio cholera...	2021	2022	6
	A00.1	Χολέρα από στο δονάκιο της χολέρας [Vibrio ch...	2022	2023	8

Σχόλια:

Η χρήση του CTE (YearlyAdmissions) είναι επιβεβλημένη, καθώς αναλαμβάνει την αρχική, βαριά ομαδοποίηση των δεδομένων ανά έτος και κωδικό διάγνωσης. Η χρήση της HAVING COUNT(...) >= 5 στοχεύει στον αποκλεισμό "τυχαίων" ή σπάνιων παθήσεων, εστιάζοντας μόνο σε στατιστικά σημαντικά δείγματα. Το πιο κομβικό σημείο του κώδικα είναι η συνένωση του CTE με τον εαυτό του (JOIN YearlyAdmissions y2). Αυτή η τεχνική επιτρέπει τη γραμμή-προς-γραμμή σύγκριση δεδομένων από διαφορετικά έτη. Η αλγεβρική συνθήκη y2.admission_year = y1.admission_year + 1 αποτελεί τον πιο κομψό τρόπο στην SQL για να ταυτοποιηθούν αυστηρά διαδοχικά έτη (π.χ. 2024 με 2025), διασφαλίζοντας ότι το πλήθος (total_admissions) συγκρίνεται μόνο μεταξύ γειτονικών χρονικών περιόδων.

Ερώτημα 15 (Q15)

Βρείτε την κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή παραπομπών ανά τμήμα.

```

WITH TriageAdmissions AS (
  -- ΒΑΣΙΚΟ ΒΗΜΑ: Ενώνουμε το Triage με τις Εισαγωγές στο παράθυρο των 24 ωρών
  SELECT
    t.triage_id,
    t.urgency_level,
    t.arrival_date,
    a.admission_code,
    a.department_id,
    TIMESTAMPDIFF(MINUTE, t.arrival_date, a.date_in) AS wait_time_minutes
  FROM Triage t
  LEFT JOIN Admission a
    ON t.patient_AMKA = a.patient_AMKA
    AND a.date_in >= t.arrival_date

```



```

        AND a.date_in <= DATE_ADD(t.arrival_date, INTERVAL 24 HOUR)
    ),
    LevelAggregates AS (
        -- 1. Υπολογισμός ΑΝΑ ΕΠΙΠΕΔΟ ΕΠΕΙΓΟΝΤΟΣ (Γενικά στατιστικά επιπέδου)
        SELECT
            urgency_level,
            COUNT(triage_id) AS total_level_cases,
            AVG(wait_time_minutes) AS avg_wait_time_level
        FROM TriageAdmissions
        GROUP BY urgency_level
    ),
    DepartmentDistribution AS (
        -- 2. Υπολογισμός ΑΝΑ ΕΠΙΠΕΔΟ ΕΠΕΙΓΟΝΤΟΣ ΚΑΙ ΑΝΑ ΤΜΗΜΑ (Κατανομή
        παραπομπών)
        SELECT
            ta.urgency_level,
            COALESCE(d.name, 'Χωρίς Εισαγωγή') AS department_name,
            COUNT(ta.triage_id) AS referrals_to_dept
        FROM TriageAdmissions ta
        LEFT JOIN Department d ON ta.department_id = d.department_id
        GROUP BY ta.urgency_level, d.name
    )
    -- ΤΕΛΙΚΟ ΑΠΟΤΕΛΕΣΜΑ: Συνδυάζουμε τις δύο βαθμίδες πληροφορίας
    SELECT
        la.urgency_level AS `Επίπεδο Επείγοντος`,
        la.total_level_cases AS `Συνολικά Περιστατικά Επιπέδου`,
        ROUND(la.avg_wait_time_level, 2) AS `Μέσος Χρόνος Αναμονής Επιπέδου (Λεπτά)`,
        dd.department_name AS `Τμήμα Παραπομπής`,
        dd.referrals_to_dept AS `Πλήθος Παραπομπών στο Τμήμα`,
        -- Το ποσοστό των περιστατικών του επιπέδου που κατέληξαν στο συγκεκριμένο τμήμα
        ROUND((dd.referrals_to_dept * 100.0) / la.total_level_cases, 2) AS `Ποσοστό Εισαγωγών στο
        Τμήμα (%)`
    FROM LevelAggregates la
    JOIN DepartmentDistribution dd ON la.urgency_level = dd.urgency_level
    ORDER BY
        la.urgency_level ASC,
        dd.referrals_to_dept DESC;

```

Αποτελέσματα:

	Επίπεδο Επείγοντος	Συνολικά Περιστατικά Επίπεδου	Μέσος Χρόνος Αναμονής Επίπεδου (Λεπτά)	Τμήμα Παραπομπής	Πλήθος Παραπομπών στο Τμήμα	Ποσοστό Εισαγωγών στο Τμήμα (%)
▶	1	39	6.85	Χωρίς Εισαγωγή	5	12.82
	1	39	6.85	Ουρολογικό Τμήμα	5	12.82
	1	39	6.85	Πνευμονολογικό Τμήμα	4	10.26
	1	39	6.85	Καρδιολογικό Τμήμα	4	10.26
	1	39	6.85	Ενδοκρινολογικό Τμήμα	3	7.69
	1	39	6.85	Οφθαλμολογικό Τμήμα	3	7.69
	1	39	6.85	Νευρολογικό Τμήμα	3	7.69
	1	39	6.85	Παθολογικό Τμήμα	3	7.69
	1	39	6.85	Ορθοπεδικό Τμήμα	2	5.13
	1	39	6.85	Χειρουργικό Τμήμα	2	5.13
	1	39	6.85	Ψυχιατρικό Τμήμα	2	5.13
	1	39	6.85	ΩΡΛ Τμήμα	1	2.56
	1	39	6.85	Δερματολογικό Τμήμα	1	2.56
	1	39	6.85	Γαστρεντερολογικό Τμ...	1	2.56
	2	42	40.75	Χωρίς Εισαγωγή	6	14.29
	2	42	40.75	ΩΡΛ Τμήμα	5	11.90
	2	42	40.75	Δερματολογικό Τμήμα	5	11.90
	2	42	40.75	Μονάδα Εντατικής Θερ...	5	11.90
	2	42	40.75	Ψυχιατρικό Τμήμα	4	9.52

Σχόλια:

Η πιο προχωρημένη τεχνική του ερωτήματος βρίσκεται στο LEFT JOIN του πρώτου CTE (TriageAdmissions). Επειδή πολλές φορές δεν υπάρχει αυστηρό Ξένο Κλειδί που να συνδέει άμεσα την εγγραφή του ΤΕΠ με την τελική εισαγωγή, η συσχέτιση γίνεται μέσω ενός χρονικού παραθύρου: η ημερομηνία εισαγωγής πρέπει να βρίσκεται μέσα σε 24 ώρες από την άφιξη στο ΤΕΠ ($a.date_in \leq DATE_ADD(..., INTERVAL\ 24\ HOUR)$). Για να υπολογιστούν σωστά τα ποσοστά, το πρόβλημα χωρίστηκε σε δύο ανεξάρτητα CTEs. Το LevelAggregates υπολογίζει τα μακρο-στατιστικά (συνολικά περιστατικά επιπέδου), ενώ το DepartmentDistribution υπολογίζει τα μικρο-στατιστικά (περιστατικά ανά τμήμα). Η τελική τους συνένωση επιτρέπει τη δυναμική διαίρεση για την εύρεση του ποσοστού επί του συνόλου ($dd.referrals_to_dept * 100.0 / la.total_level_cases$). Η χρήση της COALESCE(d.name, 'Χωρίς Εισαγωγή') αποτυπώνει με κομψό τρόπο την πραγματικότητα του νοσοκομείου: τα περιστατικά που δεν οδήγησαν σε κανονική νοσηλεία (άρα το department_id παρέμεινε NULL) ομαδοποιούνται ορθά ως εξιτήρια απευθείας από τα Επείγοντα.